# Compiling the Coordination Protocols of Multi-Agent Systems: Provably Safe and More Token-Efficient

Anonymous authors
Paper under double-blind review

**Abstract**

Multi-agent systems built from large language models (LLMs) are programmed today through natural-language coordination artifacts such as skill files and agent-definition documents. These artifacts decide who may send what to whom, in what order, and under which authorization. They are programs, yet they ship without a compiler, and testing cannot close the gap; no number of successful runs proves that a system will never deadlock, leak a confidential payload, or miss its goal, because a guarantee over all executions requires a proof, and a proof requires a type system with a formal logic. We present STJP, a compiler for these artifacts grounded in multiparty session type (MPST) theory. An LLM drafts the user's intent as a formal Scribble protocol; a static checker proves, before any agent runs, that the protocol cannot deadlock, that every message has a legal receiver, and that its goal is reachable; the proven protocol is then compiled into per-agent local contracts, deterministic monitors, branch guards, and a scheduler, which together enforce it on free-running, untrusted LLM agents at the message boundary. Across a 1,200-trial validation suite, two model families, and nine systems built from unmodified public skill files, the enforced system is simultaneously the safest and the cheapest configuration. Local contracts produced by the formal projection algorithm complete the same tasks with 63% fewer tokens, and the saving grows to 17× at ten agent roles. Protocol-driven scheduling cuts LLM calls by 73% and makes the enforced system up to 22× cheaper per violation-free success. Enforcement commits zero unauthorized irreversible acts at every model tier, where unenforced systems commit them in up to 95% of trials; the unmodified public skill files deadlock, stall, or succeed only by chance, whereas a defective protocol is rejected before a single token is spent. Every metric is computed mechanically from a typed event log with no LLM judge, and the suite is released as a benchmark. Prompted compliance is a behavior; compiled enforcement is a property, and the property costs fewer tokens.

## 1 Introduction

Practitioners now steer large language model (LLM) agents with natural-language behavioral specifications. Prominent examples include the `SKILL.md` files of Anthropic's Agent Skills open standard, the `AGENTS.md` files of the convention of the same name, OpenAI's Model Spec, the tool manifests of the Model Context Protocol (MCP), and enterprise agent-specification documents [1]; we refer to this artifact class collectively as *coordination artifacts*. Functionally, these artifacts are programs. They fix an interaction discipline—which role sends which message to which role, in what order, carrying what payload, with which authorization preceding which irreversible act. When several agents must cooperate, such as a planner, an analyst, an auditor, and a filer, the artifact is not a prompt; it is a *protocol*.

Yet the toolchain that grew around ordinary programming languages is absent. There is no parser that rejects an incoherent skill, no type checker that proves two skills cannot mutually block, no compiler that guarantees the declared goal is reachable. Format linting exists—Skilldex scores community skill packages against the published format specification and reports widespread nonconformance [2]—but format validity says nothing about interaction soundness. A `SKILL.md` can parse perfectly while describing a choreography in which the planner waits for a result the worker will never send, because the worker is waiting for an answer the planner cannot give. Nothing crashes; the system sits there, emitting tokens.

The field's current answers are all *dynamic*. Runtime guardrails such as AgentSpec [3] and Agent-C [4] intercept unsafe tool calls with rule domain-specific languages (DSLs) or temporal constraints checked by a satisfiability-modulo-theories (SMT) solver; benchmarks such as ST-WebAgentBench [5] and BeSafe-Bench [6] measure, after the fact, how often agents violated policy; LLM-as-judge pipelines score traces with a second model whose own reliability is contested (reported true-negative rates below 25% and expert agreement of 64–68% [7]). All of these *observe*

*executions*. None can certify a *specification*. Testing, as Dijkstra observed, shows the presence of bugs, never their absence.

This paper poses the question in its static form: **can we type-check the interaction protocol that these artifacts define—who may send what to whom, in what order, under which authorization—before any agent runs?** Our answer is STJP (Session-Typed skills/Judge Protocol), a compiler pipeline built on two decades of multiparty session type (MPST) theory [8, 9] and its industrial protocol language, Scribble [10]. The user states an intent; an LLM drafts a Scribble global protocol; the Scribble checker validates well-formedness statically—rejecting deadlock precursors, incoherent choice, unguarded recursion, unreachable goals—and only a validated protocol is projected, role by role, into local contracts that become each agent's lean skill. From the same projection we mechanically generate a deterministic finite-state monitor per role, value-dependent *choice guards* in the asserted-MPST style [11], and a scheduler driven by each role's endpoint finite-state machine (EFSM, the per-role automaton induced by the protocol; §3). Enforcement and evaluation are plain code: bit-reproducible, token-free, judge-free. Concurrent work obtains coordination guarantees by taking communication away from the agents—the coordination layer is generated as trusted code around opaque LLM calls [12]; STJP leaves the agents free and makes the boundary trustworthy.

**Contributions.**

1. **A reframing, made precise.** Natural-language coordination artifacts—skill files, agent definitions, specification documents—constitute an untyped protocol language; we give, to our knowledge, the first end-to-end pipeline that statically type-checks these *deployed artifacts* via MPST and enforces the result on *free-running, untrusted* LLM agents before and during execution—in contrast to concurrent work that obtains guarantees by generating the coordination layer as trusted code around opaque LLM calls [12] (§3–§4).
2. **Judge-free evaluation and the clean-goal-completion (CGC) metric.** Because every run is, by construction, a path through a validated global type, *conformance* and *goal achievement* receive mechanical definitions over a typed event log—no golden trajectory, no LLM judge. We report both the goal completion rate (GCR: the fraction of trials that reach the goal at all) and clean-goal completion (CGC: the fraction that reach it with *zero* protocol violations along the way), and grade every violation by its *consequence* on a five-level severity scale, where S stands for severity: S0 is a benign deviation such as a harmless message reordering; S1 is pure waste such as a duplicated message, which costs tokens but breaks nothing; S2 is a skipped obligation or a broken ordering; S3 is a run that never terminates; and S4 is an irreversible act performed before its authorization. We chose these five levels because they separate, in increasing order of harm, deviations that are invisible to the outcome, deviations that only cost money, deviations that break correctness, deviations that consume the entire budget, and the one class of deviation that cannot be undone; §5 gives the full definitions and the rationale. The scale is validated empirically: every attempt that contained a violation of severity S2 or worse went on to fail its goal (§5).
3. **Three findings that goal-completion rates alone cannot reveal** (§6–§7, Figs. 2–4). First, giving agents the correct plan as plain text is not enough once they act concurrently: agents whose prompts contained the full validated protocol reached the goal in every trial, yet performed the one forbidden irreversible act, filing the report before its approval, in 95% of those trials. Second, the safety of an unenforced system depends on which model runs it, while the safety of the enforced system does not: premature filings disappear as model capability rises, but the enforced system commits none at any capability tier. Third, once the projected contracts drive the runtime, the enforced system is simultaneously the safest and the cheapest: $9\times$ cheaper per successful run and up to $22\times$ cheaper per violation-free successful run than every unenforced alternative. A protocol seeded with a circular wait shows the failure class that no runtime method can rule out: the checker rejects it before any agent runs. We also report the one run in which unenforced protocol text matched the enforcement gate on outcome, and state what that result does and does not establish. Nine further cases assembled from unmodified public skill files reproduce all of the above in the wild and add a fourth finding: on protocols that loop, a textual plan can fail in both directions, deadlocking when it is absent and livelocking when it is present, while the compiled system completes the task at less than a third of the cost of either failure (§7).
4. **Guarantee transfer.** Each classical MPST theorem is restated as a concrete agent-system property with a direct safety or cost consequence (§3, Table 2): *communication safety* becomes "no agent can be sent a message it cannot handle"; *deadlock-freedom* becomes "circular waits are rejected before a single token is spent"; *session*

*fidelity* becomes "the event log is audit evidence against a proven reference"; *monitorability* becomes "cheap per-agent checks at each message boundary add up to a global guarantee, with no watcher agent"; *refinement* and *gradual typing* license, respectively, guarding branch decisions on data values and treating the LLM as the one untyped participant whose generated monitor is the safety cast at its boundary.

5. **A benchmark, released and self-tested.** We release the evaluation suite as a benchmark—and test the benchmark before trusting it: the static checker is stress-tested by seeding protocols with known defects (mutation testing), the runtime gate by 1,200 adversarial prompt-injection attacks, the model-dependence of every result by sweeps across model tiers, and reliability by $\mathrm{pass}^k$ statistics at $n{=}100$, with every prediction registered before running and graded after—including the falsified ones (§7). One pipeline step cannot be verified this way: translating the user's natural-language intent into a formal protocol passes through an LLM, so a protocol can be perfectly *valid* yet not what the user *meant*. We call this step the *seam*, and we release the instruments that turn it into a verifier-in-the-loop learning problem: a validated Scribble grammar (so a model can only emit parseable protocols), an EFSM-equivalence scorer (to grade a drafted protocol against a known-correct reference), a calibration-gated faithfulness panel (to check the protocol says what the intent said), and a deduplicated training corpus—each instrument measured before any training. The training program itself is governed by six go/no-go hypotheses, H1–H6, registered before any run so that success and failure are defined in advance rather than after the fact (§8.4 states each one and the design question it settles); the GPU training runs are the declared next step, and this paper ships a results template into which their numbers drop without restructuring (§8).

## 2 Related Work

**Where existing governance work sits.** Existing work on governing agent specifications clusters at three distinct levels of assurance, which we make explicit as a stack (the layering is our synthesis; each cited work occupies one level of it). *Level 0—does the artifact parse?* Format-conformance checking is commoditized: Skilldex scores skill packages against the published format specification with line-level diagnostics [2]. *Level 2—does runtime behavior match claims?* An active benchmark frontier: the best of 13 agents evaluated by BeSafe-Bench completes fewer than 40% of tasks while fully respecting safety constraints [6], and ST-WebAgentBench's Completion-under-Policy shows policy-respecting completion at under two-thirds of nominal completion [5]. Specification-level governance is emerging in contract form—GovernSpec organizes skills as explicit task contracts with permissions, confirmation gates, and acceptance tests [13], and the Layered Governance Architecture composes runtime enforcement mechanisms [14]—but neither can decide the *Level 1* question in between: is the interaction structure a specification defines semantically sound—can its roles deadlock, is its goal reachable, can it leak a payload? To our knowledge, no deployed system answers that question before execution; STJP is built to. Regulatory pressure points the same way: path-based analyses of runtime governance note that the EU AI Act's high-risk provisions, effective August 2026, demand demonstrably governed orchestration [15].

**Runtime enforcement.** AgentSpec [3] enforces trigger/predicate/action rules around tool calls and prevents >90% of unsafe executions in its evaluation; Agent-C [4] compiles temporal constraints to SMT checks interleaved with generation; the Layered Governance Architecture (LGA) [14] composes sandboxing, intent verification, zero-trust authorization, and audit logging. All act per execution. Rule DSLs cannot express, let alone verify, *global* interaction properties—deadlock-freedom, goal reachability, choice coherence—because these are properties of the protocol structure, not of any single call. STJP rejects the faulty specification before the first LLM call, the only point at which a guarantee can be universal rather than per-run; its runtime gate then enforces exactly the properties the static phase proved enforceable [16]. We are explicit about where the novelty lies: the session-type theory we build on is three decades deep and is assembled here, not invented—our contributions are the compiler framing (skills *are* an untyped protocol language; STJP is its missing compiler), the gradual-typing integration of an untyped LLM participant, the observation that the *scheduler* is itself a projection artifact, and the judge-free empirical program the rest of this paper executes.

**Session types and choreographies.** MPST [8, 9] and its toolchain Scribble [10] give protocols a type theory (§3); choreographic programming [17, 18] generates endpoint code from a global script by projection. An `AGENTS.md` is an informal choreography; STJP makes it formal and lets projection do the work. Prior natural-language-to-formal-specification pipelines (NL2Spec [19], PROSPER [20]) target linear temporal logic (LTL) and network protocol documents; we target a protocol algebra whose checker exists and whose theorems transfer.

**Table 1:** Positioning. STJP is, to our knowledge, the only system that both establishes guarantees *before* execution and enforces them on *untrusted* agents at runtime, with evaluation requiring neither a reference trajectory nor an LLM judge. "partial" = expressible per hand-written rule, without a global theorem. "n/a†": in Bollig et al. [12] coordination is trusted generated code and agents cannot address messages at all, so recipient legality is moot rather than checked; the guarantee does not transfer to runtimes where agents send messages themselves.

| | When it acts | Deadlock-freedom | Goal reachability | Unauth. recip. blocked | Value/branch guards | Judge-free metrics |
|---|---|---|---|---|---|---|
| LLM-as-judge | post hoc | no | no | no | no | no |
| Rule guards [3] | runtime | no | no | partial | partial | no |
| Temporal/SMT [4] | runtime | no | no | partial | partial | no |
| Graph frameworks (LangGraph) | wiring | no | no | no | no | no |
| MSC codegen [12] | compile (codegen) | static | no | n/a† | no | no |
| **STJP (this work)** | **compile + runtime** | **static** | **static** | **typed** | **yes** | **yes** |

**Projection-based coordination for LLM agents.** Concurrently with this work, Bollig et al. [12] introduce a domain-specific language based on message sequence charts (MSCs), called ZipperGen, whose syntax-directed projection yields deadlock-free local programs by construction, with the metatheory machine-checked in Lean 4—a mechanization discipline we do not match and genuinely admire; the classical MPST theorems STJP inherits are instead backed by three decades of peer-reviewed proof, including the monitoring theorem for *untrusted* endpoints [16] that the closed setting of Bollig et al. [12] does not require. The two systems are separated by the trust boundary. In ZipperGen the LLM never holds the send keys: coordination is executed by trusted generated code (framework-owned threads and first-in-first-out (FIFO) queues), and LLM calls are opaque typed functions inside it; deadlock-freedom holds because the untrusted component is structurally incapable of sending a message at all. That is a clean closed-world result—and it is precisely why it cannot govern the deployed agent ecosystem, where free-running LLM agents (skill-driven assistants, MCP tool users, group chats, graph-framework agents) *are* the message-senders. STJP targets that open regime: the LLM is the untyped participant in the sense of gradual session types [21], the generated monitor is the cast at the boundary, and the operative theorem is monitorability [16]. Every phenomenon our empirical program measures—premature filings in 95% of trials under concurrent scheduling, unauthorized irreversible acts by intent-only agents under the stronger model, 1,200 injected exfiltration attempts—exists only in the open regime; Bollig et al. [12] report no empirical evaluation. The systems also differ on *when* assurance is paid for: ZipperGen's runtime planner lets an LLM generate sub-workflows that inherit the structural guarantees, but structure is all its DSL encodes—goal reachability, payload refinements, authorization ordering, and recipient confidentiality are absent (data-level verification is explicitly deferred)—so a generated workflow that is structurally sound yet wrong in intent is discovered mid-execution, tokens already spent, with no human endorsement point. STJP front-loads every checkable property to compile time, where a rejected draft costs zero tokens and the human endorses $\mathsf{G}$ before any agent runs (§4.1); guarantees are purchased before spend, not during it. The projected artifacts differ accordingly: ZipperGen emits local Python bound to its runtime, whereas STJP emits the ecosystem's lingua franca—lean markdown contracts consumable by any agent runtime—alongside monitors that our portability experiment (E7, §7) shows agreeing across independent runtimes. We also note the automata-theoretic line of Li et al. [22, 23], Li & Wies [24] on complete and certified MPST projection, a natural upgrade path for the compiler's validation and projection stages (S2–S3, §4.1), and the trace-based assurance framework of Paduraru et al. [25], which, like all runtime observation, is downstream of the static question posed here.

**Trace evaluation.** Tool-and-step metrics (LangSmith, DeepEval, Phoenix, MLflow), process-utility metrics (TRACE [26]), policy-conditioned metrics (Completion-under-Policy, CuP [5]), and reliability metrics ($\mathrm{pass}^k$, the probability that all $k$ of $k$ independent runs succeed) all presuppose a hand-authored reference trajectory or an LLM judge, because an unconstrained agent leaves no structure to score a run against. Under STJP the validated global type *is* the reference (§5).

# 3 Multiparty Session Types: Syntax, Semantics, Guarantees

We summarize the fragment of MPST that STJP compiles to, in the modern formulation [8, 9]; this section fixes notation and states (without proof) the theorems, labeled T1–T4 ("T" for theorem), whose transfer to agents Table 2 prices.

## 3.1 Global and local types

Fix a set of *roles* $p, q, r \in \mathcal{R}$ (agents, tools, the orchestrator), *labels* $\ell \in \mathcal{L}$ (message kinds), and *payload sorts* $U$ (base sorts and refinements, below). *Global types* describe the whole multiparty interaction from a bird's-eye view; *local types* describe one role's view of it:

$$\mathsf{G} ::= p \to q : \{\ell_i(U_i).\, \mathsf{G}_i\}_{i\in I} \mid \mu t.\, \mathsf{G} \mid t \mid \mathtt{end} \qquad \text{(global types)}$$
$$\mathsf{T} ::= q \oplus \{\ell_i(U_i).\, \mathsf{T}_i\}_{i\in I} \mid p \,\&\, \{\ell_i(U_i).\, \mathsf{T}_i\}_{i\in I} \mid \mu t.\, \mathsf{T} \mid t \mid \mathtt{end} \qquad \text{(local types)}$$

$p \to q : \{\ell_i(U_i).\mathsf{G}_i\}_i$ reads: $p$ chooses one label $\ell_i$, sends it with a payload of sort $U_i$ to $q$, and the interaction continues as $\mathsf{G}_i$. Dually, $q \oplus \{\ldots\}$ is an *internal choice* (this role selects and sends to $q$) and $p \,\&\, \{\ldots\}$ an *external choice* (this role branches on what it receives from $p$). $\mu t.\mathsf{G}$ is guarded recursion (loops); `end` terminates the session. In the agent reading: labels are message kinds (`ExpenseData`, `Approval`), sorts are payload schemas, branching is a decision (audit vs. standard path), recursion is a bounded retry/deliberation loop.

## 3.2 Projection

The *projection* $\mathsf{G} \upharpoonright r$ extracts role $r$'s local type from $\mathsf{G}$:

$$\big(p \to q : \{\ell_i(U_i).\mathsf{G}_i\}_{i\in I}\big) \upharpoonright r \;=\; \begin{cases} q \oplus \{\ell_i(U_i).\, \mathsf{G}_i \upharpoonright r\}_{i\in I} & \text{if } r = p \\ p \,\&\, \{\ell_i(U_i).\, \mathsf{G}_i \upharpoonright r\}_{i\in I} & \text{if } r = q \\ \sqcap_{i\in I}\, \mathsf{G}_i \upharpoonright r & \text{otherwise,} \end{cases}$$

with $(\mu t.\mathsf{G}) \upharpoonright r = \mu t.(\mathsf{G} \upharpoonright r)$ when $r$ occurs in $\mathsf{G}$ (else `end`), $t \upharpoonright r = t$, $\mathtt{end} \upharpoonright r = \mathtt{end}$. The *merge* $\sqcap$ requires that a role not involved in a choice either behaves identically on all branches or is informed of the outcome before its behavior may differ; if the merge is undefined, $\mathsf{G}$ is *rejected as unprojectable*—already a useful lint: it flags protocols in which an agent's duties silently depend on a decision it was never told about. Concurrent MSC-based work removes this side condition by construction, inserting owner-side control broadcasts so that projection is total [12]; we keep the merge check deliberately: in the skills setting an undefined merge is diagnostic signal—it names the role whose obligations depend on a choice it cannot observe—and the repair (informing that role) changes who learns what, a design decision the human should endorse at S1, not one the compiler makes silently. A global type is *well-formed* if it is projectable onto every role and all recursion is guarded. Each local type induces a finite *endpoint finite-state machine* (EFSM) $M_r = (S, s_0, \Sigma, \delta, F)$ whose alphabet is $r$'s sends and receives; conversely, under well-formedness, the (associative) composition of the projected EFSMs recovers the behaviors of $\mathsf{G}$ [27].

## 3.3 Operational semantics and typing

A *configuration* $C$ pairs role processes with FIFO queues (asynchrony); actions $pq!\ell(v)$ / $pq?\ell(v)$ enqueue and dequeue, and $\mathrm{traces}(C)$ collects finite action sequences. The typing judgment $\Gamma \vdash C \triangleright \Delta$ assigns each role its local type; in the "Less is More" formulation [9], behavioral properties are checked directly on the typing environment, repairing soundness gaps of the classical presentation and enlarging the set of typable protocols. Well-typedness is preserved under reduction (subject reduction), which drives:

**T1 (Communication safety.)** In a well-typed configuration, no role ever receives a message whose label or payload sort it cannot handle at its current state: every send is matched by a compatible receive [8].

**T2 (Session fidelity / protocol conformance.)** Every trace of a well-typed configuration is a trace of $\mathsf{G}$: execution *refines* the global type [8, 9].

**T3 (Deadlock-freedom / progress.)** A well-typed single-session configuration never reaches a stuck non-final state: some role can always move until `end` [8, 9].

**T4 (Monitorability.)** If every endpoint is wrapped by a monitor that enforces its projected local type $\mathsf{G} \upharpoonright r$—accepting, suppressing, or rejecting boundary messages—then the observable behavior of the whole (untrusted) network satisfies $\mathsf{G}$; local checks compose to a global guarantee with no central observer [16].

T4 is the license for STJP's architecture: LLM endpoints are untrusted, monitors are generated from projections, and *global* compliance falls out of *local*, $O(1)$-per-event checking.

### 3.4 Extensions STJP uses

**Refinements and choice guards.** Asserted/refined MPST [11, 28] decorates payloads with predicates, $\ell(x{:}U)\{A\}$, where $A$ may mention previously bound values; Chen & Honda [29] extend this line to *stateful* asynchronous properties—assertions over virtual state that evolves across the whole session, checkable at endpoints under asynchrony—implemented and measured as STJP's *session ledger* (§9); and decorates *choices* with guards selecting which branch the data permits: $p \to q : \{\, \ell_i(x_i{:}U_i)\{A_i\}.\, \mathsf{G}_i \,\}_i$ with $A_i$ over the session history. Statically dischargeable guards go to an SMT check; guards over LLM-produced values compile into the monitor. This is precisely the construct that closes the "protocol-legal but value-wrong" hole we observed empirically (§6): choosing the `Standard` branch on $50k+ revenue is legal for classical MPST and a violation for guarded MPST.

**Behavioral flexibility via precise subtyping.** An agent need not follow its projected local type *exactly*: session subtyping characterizes every safe deviation, and its *precise* (sound & complete) characterization [30, 31] draws the line tightly—every subtype-conformant behavior is safe, and any behavior outside the relation fails in some context. This is the formal license behind our lean contracts (§7): a compressed contract refined by the verbose one loses nothing, and preciseness says exactly how much slack a monitor may lawfully grant before it must reject; both directions—a compile-time subtype check and a tolerance-bounded gate—are implemented and measured in §9. **Gradual typing of the LLM endpoint.** Gradual session types [21] mix statically typed participants with a dynamically typed (`dyn`) one, inserting casts at the boundary that preserve safety and progress. The LLM is STJP's `dyn` participant; the generated monitor *is* the cast. The theory predicts what we measure: the untyped participant may *attempt* anything; nothing ill-typed crosses the boundary.

**Beyond the fragment.** Parameterised MPST [32] types fan-out over $n$ workers; dynamic multirole session types [33] admit roles joining/leaving—both on our roadmap, neither exercised by the experiments below, which fix the role set.

## 4 The STJP Compiler

### 4.1 Pipeline

The compiler proceeds in five stages, which we label S1 through S5. The letter S here denotes a pipeline *stage*; the same labels appear as the numbered badges in Fig. 1, and later sections refer back to individual stages by them. These stage labels are distinct from the violation-severity grades S0–S4 of §5, which grade runtime harm rather than pipeline position.

**S1 — From intent to draft protocol.** The user states a task in natural language, for example that six agents must process a revenue report and that any revenue above $50,000 requires an audit before filing. An LLM drafts a corresponding Scribble global protocol comprising roles, message labels, payload sorts, branch points, and guarded recursion. The draft is accompanied by a sidecar file of refinement and choice guards, held in a separate `.refn` file so that the stock `scribble-java` toolchain remains untouched, and by *goal markers*, designated protocol points whose ordered satisfaction encodes completion of the task; one marker is designated FINAL. The user endorses the draft before anything downstream consumes it, so intent negotiation is human-in-the-loop by design.

**S2 — Static validation.** The Scribble checker establishes well-formedness of the draft. It verifies projectability, which requires choice coherence under the merge operator $\sqcap$; it matches every send with a compatible receive; it

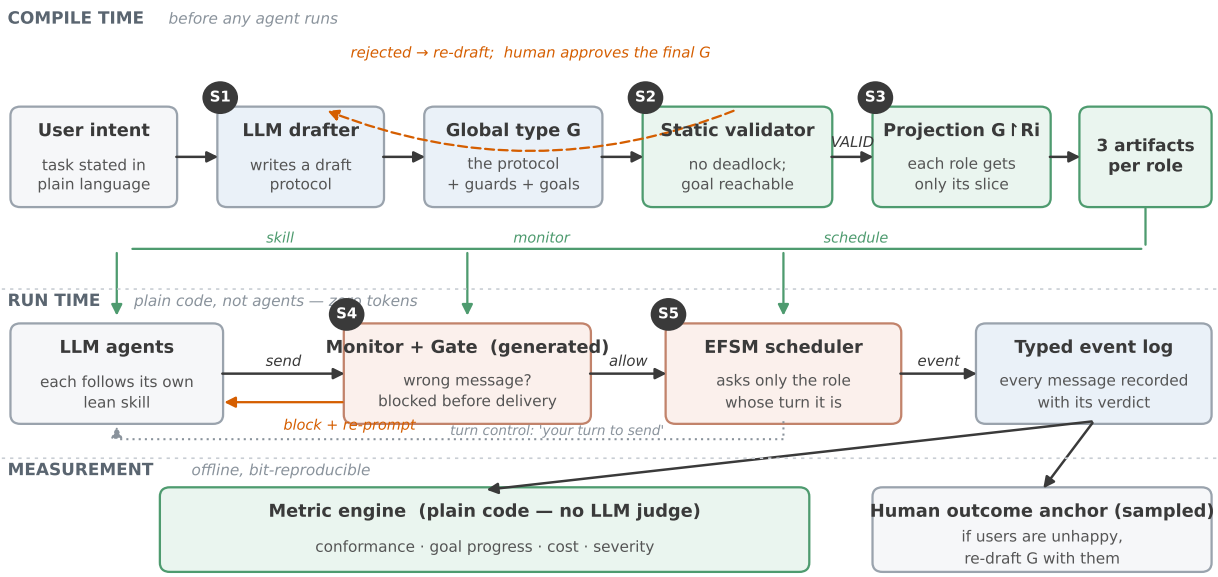


**Figure 1: The STJP pipeline**; the numbered badges are the pipeline stages S1–S5 of §4.1. *Compile time* (top row): the user states an intent and an LLM drafts it as a Scribble global type with a sidecar of value guards (S1); the static validator rejects unsafe drafts with counterexamples—drafting the banking protocol live, four consecutive LLM drafts were rejected before the fifth passed—so only a validated protocol G goes forward (S2); projection (S3) then emits three artifacts per role: a lean local contract (the agent's skill), a generated EFSM monitor (S4), and the schedule (S5). *Run time* (bottom row): the gate rejects off-contract sends *before delivery* and re-prompts the sender; the scheduler prompts only the roles whose turn the protocol says it is; every message lands in a typed event log over which all metrics are deterministic database queries. One sampled human outcome anchor stays outside the metric family.

checks that all recursion is guarded; and it confirms that every goal marker is reachable. The reachability check emits a witness path, which is later reused as the efficiency baseline. Validation is not decorative. When the banking case was drafted live, four consecutive LLM drafts were rejected before the fifth passed, and one rejected finance draft contained a wait-for cycle among the *Writer*, the *RevenueAnalyst*, and the *TaxVerifier* that would have deadlocked at runtime. The cycle was caught before any agent ran; agents executed on that unsafe draft for comparison completed only 20% of their trials.

**S3 — Projection to local contracts.** The projection $\mathsf{G} \upharpoonright r$ is rendered as a lean per-agent skill, namely a table of SEND and RECV actions over the role's EFSM states, with the role's refinement and choice guards attached at the states where each decision is made. The compressed contract is often smaller than the natural-language description it replaces; in the banking case the projected contracts span 1,063 to 1,986 characters, against 1,870 characters for the intent-only prompt. The table substitutes for coordination guesswork.

**S4 — Monitor generation.** Each local type compiles mechanically to its EFSM. The runtime monitor for role $r$ is an interpreter of $M_r$ that maintains a state pointer and a value store for guard evaluation. It is a small generated Python program, deterministic, with constant cost per event and no LLM calls. Its verdicts are *allow*, *repair*, and *block*, together with a dedicated verdict for choice-guard violations. In gate mode the monitor rejects off-contract sends before delivery and re-prompts the sending role with its enabled actions; by Theorem T4 these purely local gates jointly enforce the global type G.

**S5 — EFSM scheduling.** The same automaton knows whose turn it is. Rather than polling every agent in every round, where each idle poll is a full LLM call whose answer is merely WAIT, the scheduler prompts only the roles enabled at the current protocol state and tells a role that must send that the decision is now its own. Scheduling targets the dominant cost of the turn loop, measured at roughly 4.5 calls per delivered message when unscheduled, and the

**Table 2:** Static guarantees inherited from MPST theory, restated as agent-system properties. Each row states the general rule; concrete instances appear in the experiments of §6. The final row, for example, is what turns a branch choice that is legal for the protocol's structure yet wrong for its data—in the finance scenario, skipping a mandatory audit on high revenue—into a checkable violation.

| Guarantee | Meaning for an agent system | Consequence |
|---|---|---|
| Deadlock-freedom, T3 | A protocol containing a circular wait is rejected at compile time. | The silent-stall failure mode, in which agents wait on one another while spending tokens, is eliminated for all runs. |
| Communication safety, T1 | An agent can emit only the messages the protocol permits at its current state, and the set of legal recipients is part of the type. | Off-protocol detours and disclosure to unauthorized recipients are rejected by construction. |
| Session fidelity, T2 | Every executed trace refines the endorsed global type G. | The event log is audit evidence against a proven reference. |
| Monitorability, T4 | Constant-cost local checks at each endpoint compose to global compliance. | Enforcement is decentralized; no watching agents are needed. |
| Bounded recursion | Every loop carries a guarded bound. | Runaway loop cost is impossible, and the bound can be audited before deployment. |
| Bounded messages | Projection yields a worst-case message count for each role. | Token cost becomes a ceiling rather than a hope. |
| Minimal capabilities | The tools a role requires are exactly the message labels of its projected type. | Least-privilege tool lists are derived rather than guessed. |
| Refinements and choice guards | Payload values and branch choices are checked against declared predicates at the message boundary. | A branch choice that is structurally legal but wrong for the data becomes a detectable violation. |

dominant non-safety failure, the liveness stalls examined in §6.

### 4.2 The typed event log and judge-free metrics

The monitor emits, per message, a typed event: role, protocol position, branch choice, gate verdict, guard outcomes, goal-marker state. Over this log, evaluation is counting, not judging: conformance rate (allowed/repaired/blocked, per message—Completion-under-Policy computed mechanically); path distribution (each run reduces to a named path through G; a retry-loop entry rate jumping 9%→40% after a change is a regression no golden trajectory could encode); goal progress (read off $\checkmark\varphi$ markers—the deterministic replacement for LLM-judged "milestone completion," and the only metric class that sees an agent looping in place); detour cost (events $\div$ the compile-time witness's shortest accepting path); repair burden (gate interventions over time). All are bit-reproducible: re-running the evaluators on a run directory yields identical numbers although the measured agents are stochastic.

## 5 Evaluation Methodology

### 5.1 Defining conformance and goal achievement

**Conformance, three levels.** Let $T$ be a run's trace. *(C-struct)* Structural: every message of $T$ is accepted by the corresponding monitor EFSM—equivalently $T$ is a path through the validated G (deterministic, per-message). *(C-refine)* Refinement: all payload and choice-guard predicates hold. *(C-sem)* Semantic: payload content is faithful to the task; only this residue may require an LLM judge, run batched, sampled, calibrated on human labels, and stored apart from the deterministic verdicts. Design principle: shrink the judged surface to what is genuinely semantic.

**Goal achievement.** A run achieves its goals *strictly* iff every goal marker is satisfied, in protocol order and in the *same attempt*, including the FINAL marker, under three critical-property checks: **C1 data provenance** (reported values trace to real inputs), **C2 context completeness** (required reads precede the decision, by timestamp), **C3 authorization before irreversible acts** (`approve` precedes `file`, by message order). Relaxed rungs (role-pair, semantic) are reported alongside. Which rung serves as the headline rule is itself a fairness decision. Strict matching checks that the

exact expected label travelled the exact expected edge (did *TaxVerifier* send `Approval` to *Writer*?); an configuration that was never shown the protocol cannot know those labels, so grading it strictly is like grading an essay exam by whether the essay contains the answer key's exact sentence—the protocol configurations were handed the answer key, the bare configuration was not. A fairness audit of the harness (2026-07-17; §6.5) found the headline number doing exactly that, and the headline rule is now *per-configuration*: configurations shown the protocol vocabulary are judged strictly; bare configurations pass a goal when the right sender delivers a predicate-satisfying payload to the right receiver under *any* label (the role-pair rung); the retry loop retries on the same per-configuration rule, and both verdicts are recorded in every attempt marker. Tables in §6 that predate this fix report the strict rule for every configuration and say so in their captions; bare-configuration scores there are lower bounds, with per-configuration-rule re-runs pending. We call the fraction of trials that achieve their goals under the headline rule the **goal completion rate (GCR)**, and additionally report **clean-goal completion (CGC)**: the fraction of trials that reach the goal with *zero* violations of any severity. GCR asks "did it get there"; CGC asks "did it get there without breaking anything on the way"—the gap between them is precisely the lucky-run phenomenon that pass-rate metrics hide. One sampled human-labeled outcome measure stays *outside* the family as an anchor: a perfectly conformant system can execute a badly conceived choreography, and divergence between excellent protocol metrics and unhappy users indicts G, routing to re-drafting with the human, not to patching agents.

## 5.2 Consequence-graded violations: the severity scale S0–S4

Defining a violation as any deviation from our protocol would be circular, because a baseline that never saw the protocol would then fail by construction. We therefore proceed in two steps. First, observed message labels are aligned semantically before judging, so that the same content sent under a different label is not penalized. Second, each surviving deviation is graded by its *consequence* on a five-level severity scale, written S0–S4, where S stands for severity (distinct from the pipeline stages S1–S5 of §4.1). The levels, from harmless to catastrophic, are: **S0**, a benign deviation such as a reordering of independent messages or the same content under a different label, which affects neither cost nor outcome and is not counted; **S1**, pure waste such as a duplicated message or an idle poll, which costs tokens but breaks nothing; **S2**, a skipped obligation or a broken ordering, such as a required read that never happened, which breaks the correctness of the run; **S3**, a run that never terminates, which consumes the entire round budget; and **S4**, an irreversible act performed before its authorization, such as filing a report before its approval, which no retry can undo. We designed the scale this way because these five classes have strictly increasing consequences—invisible, costly, incorrect, budget-exhausting, and irreversible—and because the boundary that matters most in practice, between recoverable and unrecoverable harm, must be explicit in the grading rather than inferred afterwards. The grading is computed against the partial order of data and authority dependencies declared in the protocol, with each edge annotated by whether the act it guards is reversible. The scale is itself validated empirically: across every configuration of every run, every attempt that contained a violation of severity S2 or worse went on to fail its goal, while benign deviations did not predict failure—the grader measures harm, not mere difference.

## 5.3 Experimental configurations and fairness

Each experiment runs the same task under several *configurations*. A configuration is one experimental condition: the same task, the same agents, and the same model, differing only in how much of the protocol the agents are shown and whether the protocol is enforced. All configurations share the intent, role descriptions, model, step and retry budgets; the canonical validated type judges every configuration (agents in A never see it). The configurations form a ladder of increasing protocol information: **A**: intent only (the agents are told the task, nothing else). **B**: A + the full validated global type pasted as text into every agent's prompt. **C**: A + only the role's projected local contract (with the monitor observing but not intervening). **C-min**: C with a one-line-per-transition compressed contract. **C+**: C plus the *enforcement gate*, which rejects off-contract messages before delivery (and, where stated, the scheduler — the full STJP stack). Experiments were pre-registered: predictions written before running, graded after (§6.7). Three further fairness rules bind the reporting. *Uncertainty is printed*: success rates at $n \leq 10$ carry Wilson 95% intervals (the confidence interval obtained by inverting the normal test on a proportion, which stays sensible at 0% and 100%)—at $n=10$, a 0% configuration and a 40% configuration overlap ($[0, 28]\%$ vs. $[17, 69]\%$), so single-cell gaps under roughly 40 points are suggestive rather than settled. *Answer keys cannot drift*: re-anchoring goals onto a drafted protocol may relabel messages but never change pass predicates; an automated invariance check refuses any predicate change unless

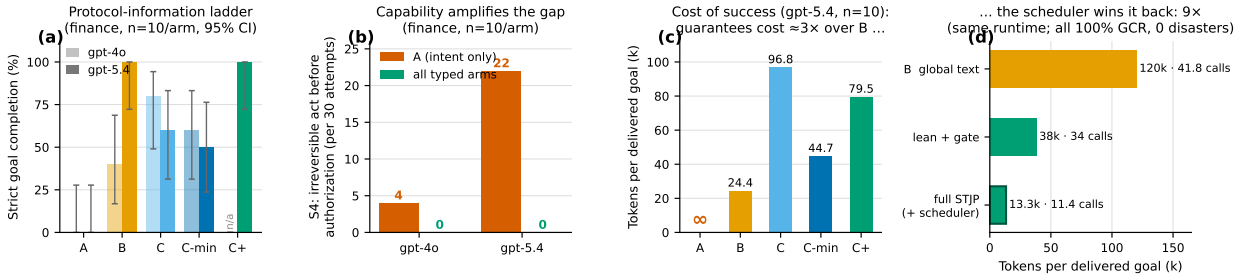


**Figure 2: Headline results on the finance scenario.** (a) Strict goal completion for both models along the protocol-information ladder—the configuration sequence A → B → C → C-min of §5, in which agents are shown successively more of the protocol. (b) Disasters, i.e. severity-S4 violations, an irreversible act before its authorization: intent-only agents under the *stronger* model committed one in 73% of attempts; every configuration that saw the typed protocol committed none. (c) Cost of one delivered goal (gpt-5.4 grand run): unenforced protocol text (configuration B) is cheapest when the model happens to comply—guarantees cost $\approx 3\times$ over B on this run. (d) The EFSM scheduler recovers that cost: on a shared decentralized runtime, full STJP delivers the same 100%-completion, zero-disaster outcome at 13.3k tokens per goal vs. 120k for protocol-as-text ($9\times$ cheaper) with 73% fewer LLM calls; tokens per call stays flat, so the entire saving comes from making fewer calls, not shorter ones.

a payload-type change forced a translation, which is then flagged for human sign-off (an earlier silent drift had eased two finance predicates; both are corrected, §6.5). *Setup cost is disclosed*: drafting the protocol and re-anchoring goals cost a handful of one-time LLM calls (cents at current rates), billed to no configuration.

# 6 Experiments and Results

**The scenario, and why it is the right stress test.** Most experiments in this section share one running scenario, chosen because it makes the cost of a coordination failure concrete. In the *finance* case, six agent roles (an Ingestor, a RevenueAnalyst, an ExpenseAnalyst, a TaxVerifier, an Auditor, and a Writer) cooperate to produce a company revenue report; policy requires that revenue above $50,000 be audited, and that the TaxVerifier's approval precede the filing of the final report. Filing is *irreversible*: a report filed before its audit or approval cannot be unfiled, which is exactly the class of failure a business cannot tolerate "occasionally." The question each experiment asks is therefore not only *did the team finish?* but *did anything irreversible happen out of order on the way, and what did each configuration cost?* Each experiment runs the same scenario under several configurations (§5)—from agents given only the task description to agents governed by the full compiled stack—so that any difference in outcome is attributable to how the protocol was communicated and enforced, not to the task or the model. A second scenario (*banking*, a five-role payment pipeline where money moves) and two purpose-built tasks (§7) widen the evidence base.

## 6.1 The protocol-information ladder (finance, gpt-4o, $n$=10)

This first experiment asks the most basic question, namely whether showing agents more of the protocol improves outcomes, and if so through which mechanism. Each step of the ladder is earned by a different mechanism, as Table 3 shows. Validation earns the first step; agents given an unsafe draft of the protocol completed 20% of their trials, while agents given the validated text completed 40%. Projection with monitoring earns the second step, raising completion from 40% to 80%. The fair indictment of configuration A, after severity alignment, is not its 184 raw violations, of which 134 were benign dialect at severity S0, meaning the same content sent under different message labels; it is that A filed the report before approval or audit four times in thirty attempts. Configuration C is not spotless either. Its two S4 violations were branch choices that were legal for the protocol's structure yet wrong for its data; the agents took the standard branch on revenue high enough to require the audit branch. This is exactly the hole that classical MPST cannot express and that choice guards close, as §3 explains. With guards compiled into the contracts and monitors, this violation class becomes detectable under the observing monitor and impossible to complete under the gate, verified by offline replay in both directions with no false positives on unevaluable guards.

**Table 3:** Protocol-information ladder, finance case, gpt-4o, $n$=10 per configuration. Metrics: *GCR* = goal completion rate; *monitor acceptance* = share of sent messages the projected contract would accept; *cost of success* = mean tokens per trial ÷ GCR, i.e. tokens spent per completed goal; *redundancy ratio* = messages emitted relative to the shortest path through the protocol that completes the task (1.0 would mean zero wasted messages). Every configuration is scored here by the *strict* label rule, which configuration A was structurally unable to satisfy (it never saw the label vocabulary, §5): A's 0% is a lower bound under the now-adopted per-configuration rule, and a fair-rule re-run is pending (§6.5). Wilson 95% intervals at $n$=10: 0%→ $[0, 28]\%$, 40%→ $[17, 69]\%$, 60%→ $[31, 83]\%$, 80%→ $[49, 94]\%$—adjacent configurations overlap; the A-vs-C gap does not.

| Metric | A intent | B global text | C local | C-min |
|---|---|---|---|---|
| GCR (strict) | 0% | 40% | **80%** | 60% |
| Monitor acceptance | 0% | 60% | **100%** | 84% |
| Tokens/trial | 15.4k | 35.4k | 33.7k | 28.2k |
| Cost of success (tokens ÷ GCR) | $\infty$ | 88.5k | **42.1k** | 47.0k |
| Redundancy ratio (events ÷ min. path) | 1.9× | 2.2× | **1.5**× | 2.0× |
| Severity S2 / S3 / **S4** | 14 / 24 / **4** | 3 / 18 / 0 | 4 / 5 / **2** | 7 / 4 / **3** |

**Table 4:** Identical contracts; the only delta is the gate plus its per-turn liveness hint—a hint-free gate configuration is pre-registered to separate blocking from hinting (§6.5). gpt-5.4, $n$=5: Wilson 95% intervals are wide (80%→ $[38, 96]\%$, 100%→ $[57, 100]\%$) and overlap. Seconds were measured with configurations sharing one rate-limited deployment (contended parallel execution) and are indicative only; tokens and calls are unaffected by contention.

| Metric | C observer | C+ gate |
|---|---|---|
| GCR (strict, same attempt) | 80% | **100%** |
| Attempts per trial | 1.40 | **1.00** |
| Delivered violations | 6 / 63 events | **0 / 53** |
| Stalled attempts (S3) | 3 | **0** |
| Tokens per trial | 114.9k | **80.9k** ($-30\%$) |
| Seconds per trial | 170.3 | **122.1** ($-28\%$) |
| LLM calls per trial | 54.2 | **37.6** ($-31\%$) |

## 6.2 Enforcement: observer vs. gate (finance, gpt-5.4, $n$=5, branch-balanced)

The question here is whether enforcement changes outcomes or merely records them. Both configurations run *identical* per-role contracts; the only difference is what the monitor does with a wrong message—the *observer* logs it and lets it through, the *gate* rejects it before delivery and re-prompts the sender. ("Branch-balanced" in the heading means trials are split evenly across the protocol's audit and standard branches, so neither configuration benefits from drawing easier cases.) The mechanism is visible in the traces (Table 4): in both configurations *TaxVerifier* repeatedly attempted to send `Approval` early. Observed, the premature send was *delivered* four times and once cascaded the session off-protocol, failing the trial; gated, the identical mistake was attempted twice, rejected pre-delivery both times, the role re-prompted—and every trial completed on the first attempt. Enforcement *paid for itself*: rejecting a wrong send is cheaper than delivering it and retrying ($-30\%$ tokens at equal cost-per-success; the $-28\%$ wall-clock comes from a contended parallel run and is indicative only, §6.5).

## 6.3 Grand five-configuration comparison (finance, gpt-5.4, $n$=10)

This is the head-to-head: all five configurations, one strong model, one run of the full finance scenario. The question is no longer whether protocol information helps, but which way of delivering it—pasted text, projected contract, or enforced gate—earns its cost. Five findings, in order of importance (Table 5). **(1) B and C+ tie on outcome—and B is cheaper.** Both reach 100% with zero harm; B does it at 24.4k tokens per success vs. 79.5k. They differ in *kind*: B relies on the model *choosing* to follow pasted text (no mechanism); C+ blocks wrong messages mechanically (it intercepted five premature `Approval` sends this run). On this model and case, enforcement was sufficient but not *necessary* for the outcome; this benchmark does not show C+ beating B—it shows the *validated protocol*, present in both, doing the safety work. **(2) A stronger model makes intent-only agents more dangerous, not safer.** gpt-5.4's

**Table 5:** All five configurations, one model, one run. "Delivered violations" = protocol-violating messages that reached their recipient, over all messages sent; "harmful attempts" = attempts containing a violation of severity S2 or worse. GCR is the strict label rule for every configuration (A's 0% is a lower bound, §5); Wilson 95% intervals at $n$=10 run from $[0, 28]\%$ at 0% to $[72, 100]\%$ at 100%. Seconds come from contended parallel execution and are indicative only.

| Metric | A intent | B global | C local | C-min | C+ gate |
|---|---|---|---|---|---|
| GCR (strict) | 0% | **100%** | 60% | 50% | **100%** |
| Attempts per trial | 3.00 | 1.00 | 1.90 | 2.00 | **1.00** |
| Delivered violations | 180/180 | 26/100 | 16/151 | 15/153 | **0/105** |
| S2 / S3 / **S4** | 29 / 1 / **22** | 0 / 0 / 0 | 0 / 13 / 0 | 0 / 15 / 0 | **0 / 0 / 0** |
| Harmful attempts | **97%** | 0% | 0% | 0% | **0%** |
| Tokens per trial | 21.7k | 24.4k | 154.1k | 84.0k | 79.5k |
| Tokens per success | $\infty$ | **24.4k** | 96.8k | 44.7k | 79.5k |
| Seconds per trial | 109 | **51** | 230 | 245 | 121 |

configuration A acted confidently and fast: it committed an unauthorized irreversible act in 73% of attempts, harmed the outcome in nearly every attempt, and completed none under the strict rule (a lower bound, §6.5), whereas the weaker gpt-4o committed such acts far more rarely. Capability amplifies the coordination gap. **(3) B's 100% is real but unguaranteed.** The same configuration scored 40% (gpt-4o), 0% (gpt-4o live) and 50%-with-stalls (banking); its good behavior is a property of the model and the protocol's shape, not of the system, and it carries no mechanism preventing the configuration A failure mode. **(4) Observer projection underperformed—from liveness, not contracts.** All 13/15 failures in C/C-min were S3 stalls; C+ uses *identical contracts* plus the gate and scheduling nudge: 60%→100%. The projection must *drive* the runtime, not just judge it. **(5) Guarantees cost** $\approx 3\times$ **tokens-per-success over B** on this run—the honest price of "cannot misbehave" vs. "happened not to misbehave"—which motivates the next experiment.

## 6.4 Scheduler ablation: winning the cost back ($9\times$)

Finding (5) above left enforcement three times more expensive per success than pasted protocol text, and this ablation asks whether the EFSM scheduler of stage S5, which the grand comparison did not yet use, wins that cost back. On a shared decentralized runtime (finance, gpt-5.4, $n$=10; run 2026-07-02), protocol-as-text costs 120k tokens per delivered goal at 41.8 calls/trial, because every role re-reads the whole protocol and idle roles are polled every round. Lean contract + gate: 38k at 34.0 calls. Full STJP (adding the EFSM scheduler): **13.3k tokens per goal, 11.4 calls, 32s/trial**—$9\times$ cheaper than B and $-73\%$ calls, at identical 100% GCR and zero disasters across all three (Fig. 2d). Tokens-per-call stayed flat (1.12k vs. 1.07k), so the entire saving is *fewer calls*: the mechanism, not verbosity effects. The gate rejected 10–12 off-contract send attempts per trial even in succeeding runs—enforcement measured as work done, not decoration. Two caveats scope this win. The 32s wall-clock comes from a contended parallel run and is indicative only; the token and call counts, which carry the claim, do not depend on the stopwatch. And the defeated opponent is round-robin poll-all—the weakest scheduler one can write; a protocol-free scheduling control ("poll whoever just received a message") is built and pre-registered, and the EFSM scheduler's claim must survive it on branching and fan-in cases, not linear pipelines (§6.5).

## 6.5 Fairness audit, corrected metrics, and pre-registered ablations

A benchmark whose headline reads "0% vs. 100%" should expect the question *did you rig the grading?*—so we audited our own harness (2026-07-17; the full plain-language review and the diff of every fix ship in the repository as `docs/BENCHMARK_FAIRNESS_REVIEW.md`) and report what it found, because the honest answer was "partly." Five findings, five fixes, two of them spawning ablation configurations whose live numbers are pending—reported here as pre-registered predictions, in the same no-invented-numbers discipline as the seam template (§8.4).

**(1) The old headline rule asked bare agents to guess secret words.** A trial passed only if the exact expected label crossed the exact expected edge. The protocol configurations were handed that vocabulary—the protocol *is* the list of expected labels—while the bare configuration was told only, in natural language, that "the tax verifier must approve the audit explicitly": a bare agent performing a perfect approval with the right content between the right roles under

its own wording scored zero, and the retry loop then burned up to three full attempts chasing labels it could not see, inflating the bare configuration's token bill as well. The fix (per-configuration rule, §5) is now the harness headline; the strict-rule tables above are labeled; and we *expect* bare-configuration completion to rise from 0% to something real on re-run. That is a feature: "bare teams succeed sometimes, enforcement succeeds always and cheapest" is more believable than a zero the grading itself manufactured.

**(2) Different configurations sat slightly different exams.** Re-anchoring goals onto the drafted protocol had also eased two pass predicates (one became a substring check `"true" in x`, which even "untrue" satisfies; one dropped a length requirement and changed receiver). Part of the drift was *forced*—the drafted protocol carries `Approval(Bool)` where the canonical protocol carries a `String`, so the canonical predicate could never pass—which is why the fix is a guard, not a ban: relabeling is free, predicate changes are errors unless a payload-type change forced a translation, and forced translations require human sign-off. The finance goal file is corrected under this guard.

**(3) The stopwatch was shared.** All configurations ran concurrently against one rate-limited deployment, so every wall-clock figure includes waiting in a traffic jam the benchmark itself created—and the configuration making the fewest calls got jammed the least, flattering exactly the configuration we advocate. The runner now has a `--sequential` mode and stamps every summary with its execution mode; wall-clock numbers from contended runs are labeled indicative throughout this paper, and comparative speed claims are withheld until sequential re-runs land. Token and call counts are unaffected by contention and carry the cost claims.

**(4) The scheduler had no cheap opponent.** Round-robin poll-all is the weakest possible scheduler, and beating it does not establish that the *protocol* is what schedules well. A protocol-free control configuration now exists (`min_llmvalid_gate_lastrecv`): identical prompt and gate to full STJP, next speaker chosen by the rule "poll whoever just received a message," falling back to the fixed circle. An offline simulation of a three-step linear flow already concedes the linear case to the heuristic (3 sends in 3 polls, zero idle asks), so the pre-registered prediction is deliberately falsifiable: the control ties full STJP on the linear `report_pipeline`, and loses on cases where the next actor depends on a branch decision, a fan-in join, or two legitimately concurrent senders (`finance_nested`, `intel_report`, `auction`), where "last receiver" has no answer. Live numbers pending.

**(5) The gate configuration also whispered hints.** In gate mode the runner, besides blocking wrong sends, tells a role at a must-send state which send is available—prevention of stalls, but also per-turn help that the observer configuration never got, so "contract vs. contract+gate" silently measured "contract vs. contract+gate+hints." A hint-free ablation configuration now exists (`min_llmvalid_gate_nohint`): rejection feedback stays (an enforcement system must say what it rejected and why); the next-move whisper goes. Pre-registered prediction: zero disasters is preserved (safety belongs to blocking), some liveness is lost (the hint exists to prevent the WAIT stalls of §6.8). Live numbers pending; until they land, gate-configuration liveness numbers should be read as gate+hint.

## 6.6 Banking: S4 in dollars; and the statically caught deadlock

On the live-drafted 5-role banking pipeline (gpt-4o, $n$=2), intent-only agents *moved money before authorization* twice in six attempts, with nothing in their own stack flagging it; typed configurations reached 100% GCR with zero S4 (and their eight raw monitor flags collapsed under grading to a single harmless S2). Separately, a two-role trading protocol seeded with the classic circular wait (Desk waits for Executor's ready; Executor waits for Desk's request) stalls every bare run forever; Scribble rejects the global type *before any agent runs*, at zero token cost, and the repaired protocol executes at 100%. No runtime guardrail, judge, or benchmark can offer this: they all require the failure to occur at least once. Testing observes runs; typing quantifies over all of them. The same circular-wait pattern is not merely authored: adapting the buyer and seller agents of a real open-source negotiation benchmark [34] (MIT) and adding an escrow settlement layer reproduces the deadlock, and a bounded live run across three model tiers shows it is capability-invariant while the validated escrow-first protocol completes (full metered benchmark pending).

## 6.7 Pre-registered predictions, graded — including the ones we got wrong

Predictions were written down before the runs and are labeled P1–P5 ("P" for prediction). **P1 (falsified twice, differently).** We predicted unenforced global text (B) would slip below 100%: under gpt-4o it did (40%); under gpt-5.4 it did not (100%, §6.3). What survives both outcomes: the gate rejected real off-contract attempts in *every* gated run—the model tries to stray; the boundary catches it—and enforcement value is a decreasing function of model strength while the danger of *no protocol at all* is an *increasing* one: given only the intent, the stronger model committed several times

more unauthorized irreversible acts than the weaker one. **P2 (confirmed beyond margin).** Scheduling saves $\geq$60% tokens vs. lean+gate: measured $-65\%$ tokens, $-66\%$ calls. **P3 (half-confirmed).** Projection alone does not beat global text; projection *with enforcement and scheduling* dominates it on the same runtime. **P4 (deferred).** Orchestrator-cost comparison was inconclusive (infrastructure issues on the group-chat configuration). **P5 (confirmed).** Scheduling cuts calls without inflating tokens per call. We report these grades because a verification paper that hides its own falsified predictions has misunderstood its subject.

### 6.8 Two systems lessons

**Liveness cannot be enforced by observation.** The dominant failure of one gate run was inaction: a role whose whole contract is one SEND answered WAIT repeatedly; a monitor judges events, and a WAIT emits none. A contract can make every action that happens correct; it cannot, by observation alone, make an action happen. The fix is structural—the projection already contains the schedule (S5). **Tone is part of the contract surface.** An imperative liveness nudge ("WAIT is OFF-CONTRACT here; act now," with a mojibaked em-dash) made gpt-5.4 *refuse outright* in both nudged roles, stalling the configuration at zero events; the same content rephrased descriptively ("Protocol status: ... the available action at this state is SEND `ExpenseData`...") was followed 5/5 first-attempt. Runtime governance text must be neutral, ASCII-safe, and informative; stronger models police the difference.

## 7 The Validation Suite at $n$=100

The pilot evidence of §6 was concept-scale ($n\leq10$, one model family, friendly agents, an untested checker). We therefore designed a component-isolating suite—one experiment per component of the system, each pre-registered before running—and executed it at $n$=100 trials per condition. The experiments are numbered E0 through E7, where E stands for experiment, and each answers one question a skeptical reader should ask. **E0** asks whether the measurement instruments themselves can be trusted. **E1** asks whether the static checker actually catches broken protocols, tested by seeding known defects in the manner of compiler mutation testing. **E2** asks whether the runtime gate withstands deliberate prompt-injection attacks that attempt to exfiltrate data. **E3** asks whether the results depend on which model is used, measured by a sweep across capability tiers. **E4** asks whether the infrastructure is reliable at scale, measured by $\mathrm{pass}^k$ statistics. **E5** asks whether the LLM's intent-to-protocol translation is faithful. **E6** asks how costs scale with the number of participating agent roles. **E7** asks whether enforcement ports to other agent runtimes. Section 9 later adds E8–E10 for the three typed extensions. The E3 capability sweep is now measured across three Claude tiers on both tasks. Everything below is **measured** and mapped number-by-number to run data in the repository; every trial ran to termination after the forensic audit; and the total compute cost of the suite was under $100.

**E0 — Testing the testers first.** *Why.* Every number in this paper is produced by the monitor and the severity grader; if that referee is wrong, everything downstream is wrong, so the referee is tested before it is used. *How.* We built a corpus of 40 traces whose correct verdicts were derived by hand: clean traces, every severity class, concurrent interleavings on independent channels, traces whose guards cannot be evaluated and must be passed over in silence, and the `Approval(False)` edge case, in which an explicit refusal must not be counted as an approval. *Result.* Monitor and grader score 40 of 40—and, instructively, three of the *hand derivations* were themselves wrong and had to be corrected against the formal semantics; the instruments were right and the humans were not. All downstream experiments inherit this validated referee. An integration stress harness additionally exercises all five pipeline stages 100 times, with 2,105 of 2,110 checks passing; the five failures all come from one mutation that swaps two independent messages and thereby yields *another* well-formed protocol, which Scribble correctly accepts.

**E1 — Mutation testing the checker (Fig. 3a).** The static checker is the load-bearing instrument of the whole pipeline, so E1 tests it the way compilers are tested. We take a corpus of 100 valid protocols spanning two to ten roles and four topologies, corrupt each with seven *mutation operators*—systematic ways of planting a known defect, such as deleting a role declaration or flipping who makes a choice—and measure whether the checker catches the planted defect without flagging the healthy originals. The pre-registered prediction separated two defect families, and the data respected the split. *Well-formedness* defects, the family the checker must catch, are detected at 95.1% across 226 mutants, with undeclared-role and flipped-choice-subject mutations caught at 100% and branch asymmetry—a role acting on branch knowledge it cannot have—at 82.5% raw, and with zero false alarms on the 100 valid originals. Adjudicating the eleven surviving branch-asymmetry mutants sharpened the picture: seven turned out to be

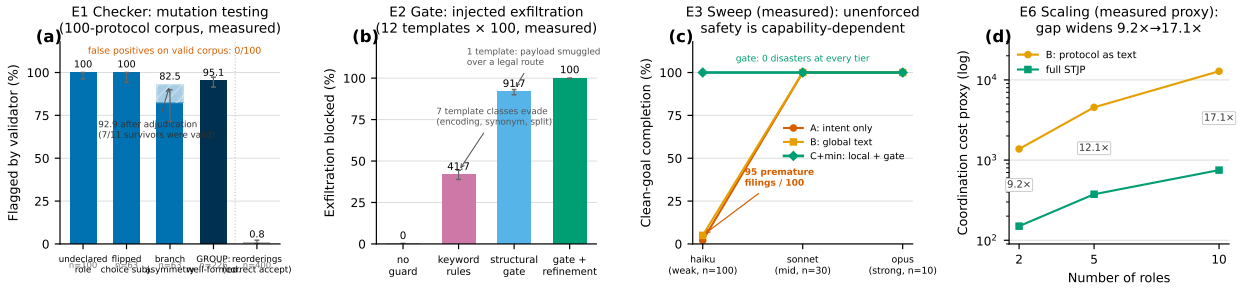


**Figure 3: Validation suite.** (a) E1, measured: detection of seeded defects over a 100-protocol corpus. Well-formedness defects (undeclared role, choice-knowledge violations) are what the checker must catch: 95.1% group detection, *zero* false positives on the valid corpus. Reorderings of acyclic protocols mostly yield *another valid protocol*; flagging only 0.8% of them is correct acceptance, not a miss—the greyed bar is a feature. (b) E2, measured: 1,200 injected exfiltration attempts per condition. Keyword rules are evaded by seven template classes (encodings, synonyms, splitting); the structural gate is disguise-proof for every illegal route but is blind to a payload smuggled inside a legitimately-labelled message to a legal recipient (one template, 8.3%); adding value-refinement guards closes exactly that hole: 100%. The layering is the finding. (c) E3, measured: clean-goal completion across three Claude tiers on `revenue_audit` (haiku $n$=100, sonnet $n$=30, opus $n$=10). Observe-configuration safety and cleanliness are capability-dependent (95 premature filings at the weak tier; zero above); the gated configuration is flat at 100% with zero disasters at every tier. (d) E6, measured structural cost proxy (contract size × scheduled polls): the global-text-to-STJP ratio grows from 9.2× at two roles to 17.1× at ten.

*accidentally valid*, meaning the mutation happened to produce another correct protocol, so accepting them was right; the remaining four share one reproducible genuine gap, in which the mutation empties a branch, projection silently drops it, and an idle role is left waiting forever for a message the choosing role may never send—a knowledge-of-choice deadlock the checker misses. Re-scoped to real defects, detection is 92.9%, and the bare percentage becomes a named limitation. *Reordering* operators (400 mutants) are flagged at only 0.8%—correctly, because reordering an acyclic protocol usually yields a different but still-valid protocol; detecting *intent* drift under valid reorderings is the E5 scorer's job, not the type checker's, and conflating the two is how benchmarks overstate their tools. The evaluation standard is deliberately the two-sided *preciseness* discipline of Chen et al. [30, 31]—soundness, nothing bad accepted, and completeness, nothing good rejected, measured together.

**E2 — Adversarial exfiltration (Fig. 3b).** *Why.* Prompt injection is the agent ecosystem's headline attack: poisoned input persuades an agent to leak what it holds. A structural claim—the type fixes who may receive what—ought to be worth something against it, and E2 measures how much. *How.* Twelve prompt-injection template classes, at 100 trials each, attack a finance protocol carrying a confidential payload; an ExternalAuditor role may legally receive only the public summary, and the attacker's goal is to route the confidential payload to it. *Result.* With no guard, 0% of attempts are blocked. An honestly tuned keyword-and-role rule list blocks 41.7% and is evaded by encodings, synonyms, typos, split payloads, and innocuous relabeling. The structural gate blocks 91.7%: it stops every attempt to use an illegal label-and-recipient pair regardless of disguise, because legality is checked against the type rather than the wording. The surviving 8.3% is exactly one template, which smuggles the payload *inside* a legitimately labelled message on a legal route—the one thing structure cannot see; adding value-refinement guards closes exactly that hole, reaching 100%. *Insight.* Each mechanism catches precisely the violation class the theory says it catches; the layering, not any single layer, is the finding.

**The six-configuration ladder at $n$=100 (Fig. 4).** The pilot's sharpest claims deserved more than ten trials each, so we built two purpose-designed tasks and ran them at $n$=100 per configuration—1,200 trials in all—off the Azure AI Foundry hosting platform, with cheap subagent role-players. Each task isolates one axis. `revenue_audit` (3 roles) probes *safety*: its single binding rule is that filing must wait for approval, so a run that files early is an unambiguous disaster. `escrow_trade` (4 roles) probes *cost*: a trade settles through an escrow, a neutral third party that holds funds until both sides deliver, so the protocol is longer, multi-round, and rewards efficient scheduling. The six configurations turn three independent knobs: *what the agent sees* (intent / global text / local projection), *whether the monitor enforces* (observe vs. gate—the same program in two modes), and *who gets polled* (all roles every round vs. only the enabled sender). Three results settle questions the pilot left open.

**(i) Unenforced protocol text no longer matches the gate.** In §6.7, unenforced global text tied the gated con-

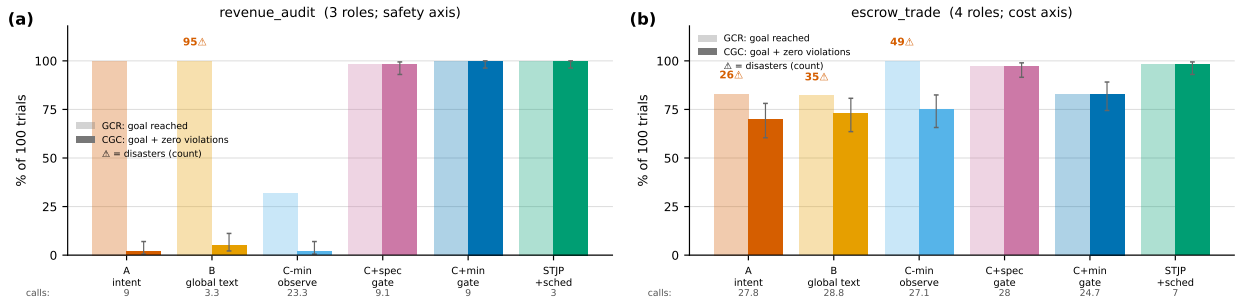


**Figure 4: The six-configuration ladder at $n$=100 (measured; 1,200 trials total, cheap subagent roles, cost in LLM calls).** Light bars: goal completion (GCR); solid: clean-goal completion (CGC, goal reached with zero violations); red counts: trials containing an unauthorized irreversible act; bottom row: mean calls per trial. (a) `revenue_audit` (file only after approval): unenforced global text reaches the goal in 100/100 trials—and files before approval in 95 of them; the observe-mode local-contract configuration collapses to 32% on pure liveness (an agent re-sending into silence); every enforcing configuration shows zero disasters *by construction*. (b) `escrow_trade`: violations invisible at $n$=10 surface in every observe configuration (26–49); after 18 abandoned-but-healthy C+spec trials were driven to termination the gated configurations tie STJP on liveness, and the scheduler delivers the cost win: 7.0 calls/trial vs. 24.7–28.8. Post-audit final numbers; 0 non-terminal trials.

figuration on outcome, and we declined to claim enforcement was necessary. At $n$=100 under concurrent polling, configuration B reaches the goal in every `revenue_audit` trial while committing the forbidden act, filing before approval, in 95% of them. Trace forensics make the mechanism exact: in 60 of those trials the Auditor *approved without ever receiving the revenue figure* and the Filer filed in the same first round; in 35 more all three messages fired in round one; only 5 trials serialized. The filed-at-round histogram is binary: B files in round 1 in 95% of trials, while the intent-only configuration files in round 3, after approval, in every trial that files at all—handed the whole protocol under concurrent polling, the weak model races through it, and denied the protocol, it fumbles slowly into a safe order by accident. The goal-completion rate conceals this entirely; clean-goal completion, which requires the goal *and* zero violations, reads 5%. A protocol communicated only as text in the prompt is not reliably obeyed once agents act concurrently: enforcement, not wording, is what carries the ordering guarantee.

**(ii) Liveness, again, structurally.** The observe-mode local-contract configuration completes only 32% of `revenue_audit` trials—all 68 stalls show the same pattern, an agent re-sending `Revenue` into an unenforced wait—reproducing at scale the lesson that contracts constrain what happens but cannot make anything happen; the projection must drive the runtime.

**(iii) Safest and cheapest, simultaneously.** Every enforcing configuration shows zero disasters at $n$=100 in both tasks—not zero-so-far but zero *by construction*, since the monitor in enforcing mode rejects the disallowed send before delivery—and full STJP is also the cheapest configuration (3.0 and 7.0 calls/trial vs. 3.3–28.8), because the scheduler never spends a call on a role whose turn it structurally is not. On `escrow_trade`, once eighteen abandoned-but-healthy C+spec trials were driven to termination (see the audit below), the gated configurations tie STJP on liveness (97–98%, all at 0 disasters); STJP's escrow advantage is then *purely* the 4× cost edge—an honest decomposition we prefer to an inflated gap.

**(iv) Reconciliation: no unenforced configuration is safe across conditions.** Holding the finance run and the ladder side by side, the failing configuration *moves*: intent-only was the villain under a strong confident model (22 disasters, finance), unenforced global text under concurrent polling (95 disasters, `revenue_audit` — trace-verified: `Filer→Analyst:Filed` in round 1 with no `Approval` preceding it, because poll-all hands the Filer its first turn before approval can exist), and the observe-mode local contract on liveness (31%) or violation volume (49, `escrow_trade`). Which unenforced configuration fails is a function of model, task, and scheduling that nobody can predict in advance—and the measured capability sweep (Fig. 3c) closes the model axis: on identical protocol text, B's premature filings go 95→0→0 across the haiku/sonnet/opus tiers, intent-only's cleanliness goes 2%→100%, and the escrow sweep mirrors both (observe-configuration disasters 26–35→0). The enforcing configurations are the only *invariant* of the whole grid—zero disasters in every task, model tier, and scheduler tested, with zero gate rejections even needed at the stronger tiers. Notably, the strong model's *safe* B costs 9.0 calls/trial against reckless haiku's 3.3: the unenforced configuration pays for safety in serialization, while STJP gets safety and 3.0 calls simultaneously because the scheduler neither races nor idles. Table 6 prices this: dividing mean calls by CGC, B's apparently cheap

**Table 6:** Cost to *clean* goal (mean calls/trial ÷ CGC), $n$=100 per configuration: the expected number of LLM calls an operator pays per violation-free success. GCR-based cost rankings reverse once violated runs stop counting as successes. Configurations as in §5: C-min obs. = minimal local contract, monitor observing only; C+spec / C+min = full / minimal contract with the enforcing gate; STJP additionally adds the EFSM scheduler.

| Task | A intent | B global text | C-min obs. | C+spec | C+min | STJP |
|---|---|---|---|---|---|---|
| `revenue_audit` | 450 | 66 | 1165 | 9.3 | 9.0 | **3.0** |
| `escrow_trade` | 39.7 | 39.5 | 36.1 | 28.9 | 29.8 | **7.1** |

**Table 7:** E4, measured: reliability at $n$=100 (infrastructure trials; Wilson 95% confidence intervals, CI). Narrowing the CI is what $n$=100 buys: $\text{pass}^{10}$ at the CI floor rises 17.6×. $\text{pass}^{10}$@LB = probability that ten of ten independent runs all succeed, computed at the interval's lower bound (LB).

| Configuration | succ/$n$ | 95% CI | $\text{pass}^{10}$@LB |
|---|---|---|---|
| unchecked, $n$=10 | 0/10 | [0.0, 27.8]% | 0 |
| STJP, $n$=10 | 10/10 | [72.2, 100]% | 0.039 |
| unchecked, $n$=100 | 0/100 | [0.0, 3.7]% | 0 |
| STJP, $n$=100 | 100/100 | [96.3, 100]% | **0.686** |

**Table 8:** E5: translation-fidelity instrument, measured; LLM-dependent measures pending.

| Measure | Result |
|---|---|
| EFSM-equivalence scorer accuracy | 300/300 |
| identical / reformatted / mutant | 100% each |
| First draft passes validator | PENDING |
| Repair rounds to validity | PENDING |
| Guard sidecar co-emission | PENDING |

3.3 calls/trial becomes 66 calls per *clean* goal once poisoned successes stop counting, and full STJP wins by up to 22× on the metric an operator actually pays—while being the safest configuration rather than trading safety for it. In dollars (weak-tier roles at list price): a clean, safe audit costs $0.38 under STJP versus $1.12–$9.09 for the alternatives, and the entire 1,200-trial suite cost under $100.

**E3 — Capability sweep (measured; Fig. 3c).** *Why.* The obvious objection to enforcement is that better models will not need it; E3 measures that objection directly rather than arguing with it. *How.* Both ladder tasks are re-run across three Claude capability tiers (haiku $n$=100, sonnet $n$=10–30, opus $n$=10; one mind per trial, every trial verified against raw state). Three findings. *Unenforced safety is capability-dependent*: B's premature filings fall 95→0→0 on identical protocol text, and escrow's observe-configuration disasters fall 26–35→0. *Unenforced cleanliness is capability-dependent*: intent-only rises from 2% to 100% CGC as duplicate-send waste vanishes—a stronger model self-organizes without being shown the protocol. *Enforcement is capability-invariant*: the gated configuration is 100% CGC, zero disasters, zero gate rejections, at every tier. As capability rises the unenforced configurations approach the gated configuration; read naively, that says strong models need no enforcement—read correctly, the closing gap is an *average* while the gate's zero is a *guarantee*: production does not get to assume the strongest model, and even strong models carry a failure tail that enforcement makes exactly zero. The one remaining E3 gap is vendor diversity (a non-Claude frontier point), an access limitation we report rather than fake. The shape of this debate is not new: every type system in history has faced "skilled programmers don't need it," and the answer adopted by every safety-critical industry is that averages improve with skill while tails do not vanish, and criticality is priced on the tail. The designed probe for the strong-model tail already exists in this suite: E2, where the violation is produced by the attack surface rather than by model weakness; running E2 with frontier-tier agents is the experiment that makes the tail visible at the top of the capability curve, and it is first on our post-submission list.

**E4 — Reliability (Table 7).** *Why.* A claim of "zero failures" is only as strong as the number of trials behind it: at $n$=10 a perfect score is statistically compatible with a system that fails a quarter of the time. E4 buys the statistical power to say more. *How.* The $n$=100 runs use a contract-following strategy in place of a live model: they certify the *infrastructure*—that the gate rejects exactly the illegal sends, the scheduler polls exactly the enabled role, and the monitor emits clean traces—at scale, while live-model behavior is carried by the finance runs (§6) and the subagent ladders above. *Result.* Under that scoping, the unchecked configuration's interval narrows to $[0, 3.7]\%$—structurally incapable, not unlucky—and the STJP configuration's $\text{pass}^{10}$ at the Wilson floor rises from 0.039 to 0.686. The $n$=100 deadlock replication is the same story in miniature: the unchecked configuration deadlocks in 100/100 trials, always in round two, burning 800 calls; STJP completes 100/100 at the protocol-optimal 7.0 calls/trial—*fewer total calls than the configuration that never finishes*.

**E5 — Translation fidelity (Table 8).** *Why.* The pipeline's front door is an LLM translating the user's intent into

a protocol; if that translation drifts, everything proved downstream is proved about the wrong protocol. Measuring drift first requires an instrument that can decide whether two protocols mean the same thing. *Result so far.* That deterministic half is done: the EFSM-bisimulation scorer—a state-machine equivalence check under which two protocols are equivalent exactly when they behave identically in every context—classifies 300 of 300 test pairs correctly, across identical pairs, semantics-preserving reformattings, and seeded mutants, when judging drafts against an expert-written reference protocol. The LLM-dependent half—first-draft validity, repair rounds, guard co-emission—remains the declared open piece of the seam and is measurement-pending; the scorer's validation is what will make those forthcoming numbers trustworthy.

**Training the seam is possible, not merely a stated goal.** Because the target grammar, the well-formedness validator, and the EFSM-equivalence scorer (Table 8) already exist, intent→protocol translation is a verifier-in-the-loop learning problem with a free, deterministic reward. We do not leave that observation as a promissory note: §8 reports the training program built on it—the system under training, the instruments (grammar, corpus, faithfulness panel, real-skills mining) measured before any weights move, and the preregistered gates whose GPU outcomes are this paper's declared next step and fill the pending E5 cells above through the shared results template.

**E6 / E7 — Scaling and portability.** E6 asks how coordination cost grows as protocols involve more participating agent roles—the practical question behind "will this still pay off for a ten-agent team?" Sweeping generated protocols from two to ten roles (Fig. 3d) confirms the pre-registered shape on a structural cost proxy (contract size $\times$ scheduled polls): with the protocol pasted as text, coordination cost grows super-linearly, because every role re-reads a protocol that grows with the team and every role is polled every round; under STJP it grows near-linearly, because each role sees only its own projected slice and only the enabled role is polled. The cost advantage of the compiled system therefore grows with team size, from $9.2\times$ at two roles to $17.1\times$ at ten; token-metered curves are pending. E7 asks whether enforcement survives leaving our own harness or is an artifact of it: the standalone generated monitor agrees with the in-process monitor on 59/59 testable canonical traces (41 choice-only protocols yield no meaningful canonical trace; the claim is scoped to testable ones), and a LangGraph adapter executing the ladder protocols as StateGraphs agrees with the native monitor on 14/14 clean-and-injected checks, consistent with T4's placement of enforcement at message boundaries; the skill-runtime third harness is pending.

**Real skills, run live: nine cases and a livelock (Table 9).** The mining result above measures what authors *write down*; live execution measures what the written-down artifacts *do*. Nine multi-agent cases were assembled from real, unmodified public sources—Anthropic's skills repository, GitHub's `awesome-copilot` collection, and the OpenAI Agents SDK, LangGraph, AutoGen, CrewAI, and AgenticPay example corpora, every source file deep-linked with license provenance—and each was executed through the trial engine under the three canonical conditions: the found files as downloaded (*no plan*), the corrected coordination plan pasted into every skill as plain text (*plan as text*), and the full compiled stack (contract + gate + scheduler), with every role played by an independently spawned subagent. Four findings.

**(i) Found files never coordinate reliably.** Seven of the nine cases never complete under the unmodified files—every trial ends in deadlock or a stall—and the remaining two complete only as a model-dependent coin flip whose direction *reverses* between Claude tiers: on 120 two-model trials, Haiku ships the announcement 10/10 but the code change 0/10, and Sonnet exactly flips. Swapping in a stronger model does not supply the missing structure; it relocates the failure—the E3 capability lesson, replicated on found artifacts.

**(ii) Plan-as-text completes without complying.** Where the textual plan reaches the goal it does so with the violation load the ladder predicts: every airline trial writes the seat twice, every booking trial charges the customer twice, and the multi-round cases emit over a hundred rule-breaking messages per run.

**(iii) Looping protocols expose a failure mode the acyclic suite could not: livelock.** The corrected code-review case (`pr_review_merge`: a `rec` loop with nested `choice` at two concurrent reviewers, ending in a two-approval join at the Merger) is the first live run of a recursive protocol through the engine, and on it the plan-as-text configuration does not merely fail—it enters a *stable livelock*: all 10 trials burn the entire 16-round budget in motion, emitting 530 rule-breaking messages and $\sim$42k tokens per trial while merging nothing, at $3.6\times$ the token cost and $5.3\times$ the LLM calls of the STJP configuration that merges 10/10 with zero violations in 7 rounds. Its sibling (`doc_coauthor_ship`, the reader-test refinement loop from Anthropic's `doc-coauthoring` skill) completes under text but with 220 violations, while STJP is clean, loops the reader test more than once before shipping, and is still $\sim 40\%$ cheaper. Deadlock is silence and livelock is motion without progress; the same static object rules out both, because both are properties of the protocol, not of any message.

**(iv) The checker earns its keep on the way in.** The corrected review protocol was itself debugged statically: two

**Table 9:** Real-skills live runs: nine cases built from unmodified public agent/skill files, three configurations each. "H"/"S" = Claude Haiku/Sonnet tiers in the two-model sweep (120 trials); *viol.* = rule-breaking messages. The `pr_review_merge` text configuration is the measured livelock: 0/10 at $3.6\times$ the cost of the succeeding STJP configuration.

| Case | Source | No plan | Plan as text | Full STJP |
|---|---|---|---|---|
| `airline_seat` | OpenAI Agents SDK | 0/10 deadlock | 10/10, 10 dbl. writes | **10/10, 0, cheapest** |
| `booking_saga` | LangGraph | 0/10 stall | 10/10, 10 dbl. charges | **10/10, 0** |
| `code_execution` | AutoGen | 0/10 deadlock | 10/10 | **10/10, cheapest** |
| `content_pipeline` | CrewAI (pattern) | 0/10 stall | 10/10 | **10/10, cheapest** |
| `doc_pipeline` | anthropics/skills | flip: H 10/10, S 0/10 | 20/20, 120 viol. | **20/20 both, 0** |
| `pr_merge` | awesome-copilot | flip: H 0/10, S 10/10 | 20/20, 120 viol. | **20/20 both, 0** |
| `agenticpay_settlement` | AgenticPay | deadlock, all 3 tiers | — | **100%, all 3 tiers** |
| `pr_review_merge` | awesome-copilot | 0/10 deadlock (round 2) | **0/10 livelock**, 530 viol. | **10/10, 0,** $3.6\times$ **cheaper** |
| `doc_coauthor_ship` | anthropics/skills | 0/10, budget exhausted | 10/10, 220 viol. | **10/10, 0,** $\sim$**40% cheaper** |

Scribble rejections ("inconsistent external choice subjects"—a role that could not tell which branch the team had taken; "role progress violation"—a role that could go rounds without a message) preceded validation, the §3 diagnostics firing on a realistic review loop before any agent ran.

Two honesty notes: the earlier casts of two of these cases had misread their sources (the `address-comments` agent presupposes an *existing* PR; the brand-guidelines skill contains no approval language), and the corrected cases move the loop and the gate to where the source files actually put them; and the runs are $n$=10 per configuration with round-batched role-play, so trials of one role within a round share a model context and are not fully independent.

**Pre-registration, graded; and the audit that changed a number.** All eight v2 predictions were registered before running and graded after: E0, E1 (with the reordering-acceptance clause), E2 (with the predicted legal-route smuggle), E4, E5-scorer, E6-proxy, and E7-agreement CONFIRMED; E3, registered pending, is now measured with its capability clause confirmed and vendor diversity still open. A post-hoc forensic audit of the full dataset then re-verified every suspicious pattern: B's low-diversity replies were adjudicated genuine on three independent behavioral signals (three distinct trace shapes, including a five-trial serialized branch no script would produce); intent-only's zero disasters trace to accidental serialization, its 591 violations all named duplicate-send waste; and the audit surfaced one material bug the checklist had not targeted—22 of 1,200 trials were abandoned mid-play rather than failed, 18 of them depressing one gated configuration's reported liveness to 79% against a true $\sim$97%. All 22 were driven to termination and every table regenerated; the 19 *genuine* gated-configuration deadlocks were left standing, because replaying real failures until they pass is p-hacking. We also log the integrity incidents scale surfaced: subagent role-players caught writing auto-responder scripts instead of reasoning per poll (detected by a `malformed==calls` signature and self-confession, discarded and replayed), a round-numbering bug that could mask disasters (fixed with causal detection), and a false-positive disaster check on enforcing configurations (fixed mid-analysis); the final audit verified all 1,200 ladder trials against raw state files with zero remaining fraud and zero non-terminal trials. We report these because a verification paper that does not verify its own experiments has misunderstood its subject.

# 8 Training the Intent-to-Protocol Translation Step, Realized

The one part of STJP's own pipeline it cannot type-check is its own front door: intent→Scribble passes through an LLM, and a *valid-yet-unintended* protocol survives the validator (§10). §7 argued this translation step (the seam) is trainable; this section reports the program that realizes the argument—a system under training, a set of instruments measured before any weights move, and a preregistered set of go/no-go gates. Consistent with the rest of this paper, everything below is either *measured today* or *explicitly pending*: the GPU training runs are the declared next step, and their numbers enter through a results template (§8.4) rather than by rewriting a table.

## 8.1 Two axes, treated differently

Translation quality has two axes with opposite epistemics. *Validity*—well-formedness, projectability, guarded recursion, guard satisfiability—is decided by the real Scribble-java toolchain, a total, counterexample-producing oracle: a draft is well-formed or it is not, and when it is not the checker returns the offending role or cycle. *Faithfulness*—does

G encode what the user meant—is not machine-decidable and is the residual judged surface (§5). The program treats them accordingly. Validity is a free, deterministic reward that can drive training directly; faithfulness is measured by a memoryless judge panel that must itself pass a calibration gate before it may reward or gate anything, and until it does the reward is verifier-only. This is the paper's judge-free discipline carried into training: shrink the judged surface to what is genuinely semantic, and never let an uncalibrated judge touch a number.

## 8.2 The system under training and its verifier reward stack

Two artifacts share one open-weights base (a 7B code model, LoRA—Low-Rank Adaptation, a lightweight fine-tuning technique that trains a small add-on set of weights instead of the whole model—heads differing): a *translator* T (intent → `.scr` + `.refn` guard sidecar) and a *repairer* R (intent, broken G, validator counterexample → fixed G); at inference they run the same draft→validate→repair loop the live drafting already exhibited (four rejected drafts before the fifth passed, §4.1). The reward is a verifier stack, not a preference model: grammar-constrained decoding (GCD—restricting the model's sampling so that only output conforming to the Scribble grammar can be produced) removes syntactic invalidity by construction; the validator contributes a pass reward; a *cheap* canonical-EFSM equivalence proxy (roles matched, transition-label multiset overlap, branch/recursion skeleton) contributes graded equivalence-to-gold, with full bisimulation reserved for evaluation because we measured it at 10–40× the cost of a validation—too slow for a rollout loop; guard co-emission is rewarded; and the faithfulness term stays exactly zero until the panel's calibration gate passes. The training recipe is deliberately standard—verifier-filtered sampling into supervised fine-tuning (SFT; the expert-iteration scheme of Zelikman et al. 35), then reinforcement learning (RL) from verifiable rewards using Group Relative Policy Optimization (GRPO) [36] with the post-R1 fixes for length bias and entropy collapse [37, 38, 39], and back-translation from a generator we own [40, 41]—because the novelty is the target and its total checker, not the optimizer. One caveat we take seriously and preregister around: constrained decoding can suppress semantic quality, not merely syntax [42, 43], so grammar-on vs. grammar-off is a primary two-sided test (H1 below), not an assumption.

## 8.3 The instruments, measured before any training

The four instruments the program needs already exist and are measured against the real toolchain (Table 10). The grammar admits exactly the corpus and nothing malformed. The corpus builder expands and EFSM-deduplicates families with a canonical signature whose verdicts agree with the pairwise equivalence checker on every tested pair, and splits them by structural family so no paraphrase or mutant straddles the train/test line. The faithfulness panel, on its first live run, did the two things it was built to do: it rejected a swapped intent/protocol canary (a planted check item with a known correct answer, mixed in to verify the panel itself is working) at 0.99, and—on `trade_deadlock`, whose intent is deliberately deadlock-shaped and whose gold protocol implements the deadlock-*free* linearization—both forward seats accepted the protocol as faithful (0.88, 0.82) while the blind back-translation seat, which never sees the intent, reconstructed a strictly linear happy-path and scored 0.25 against the original. The class structure caught a confirmation bias the forward seats shared: the protocol *repairs* the intent rather than encoding it, a policy question that escalates to the human gate, not a vote.

Finally, the real-skills mining run is a null result reported with its controls and its follow-up. The deterministic (no-LLM) path yielded zero of 13 mined teams surviving compaction into a coordinating protocol; a synthetically authored team whose skills state their interaction structure explicitly passes the same pipeline end to end, locating that failure in the inputs, not the pipeline. We then ran the disambiguating follow-up—careful model-read extraction under an evidence-only discipline (every recovered interaction must quote the skill text) over the same 13 teams—to separate *structure absent* from *structure implicit in the wording*. Reading harder recovers some structure but little: 3 of 13 teams yield a Scribble-valid protocol and a 4th surfaces a genuine deadlock the compatibility check correctly rejects, while 7 of 13 have nothing to surface. Decisively, the recoveries come from this project's own worked examples and one orchestrator file, whereas the teams built from unmodified upstream GitHub files—the only directly comparable real-world inputs—recover *zero*: ordinary skill authors do not write down interaction structure even when a human-discipline reader tries hard to find it.

A caveat the follow-up itself exposed—the harvester grouped files into "teams" without first checking that the underlying task *requires* multi-party coordination (six of the thirteen did not)—became the next experiment: we built that task-level filter and re-ran the harvest at scale. Across seven public sources (923 harvested artifacts, 52 from an

**Table 10:** The seam instruments, measured before any training, against the real Scribble-java toolchain. Corpus and repair counts are from bounded runs whose saturation curves project to the plan's targets; the reachable numbers (and their continue-commands) are reported rather than padded.

| Instrument | Measured today | Role |
|---|---|---|
| Scribble grammar (GCD) | 100% corpus round-trip (113/113 non-skip `.scr`; the 5 skips are rejected LLM drafts, correctly unparseable); 1,000/1,000 sampled protocols parse; 0 parse-level rejections under real Scribble | syntactic reward = 0 by construction |
| Protocol corpus | 671 EFSM-deduped families (bounded run; $\geq$5k reachable); canonical signature vs. pairwise checker 200/200 agreement; 860 repair tuples + 1,746 calibration near-misses; family splits leakage-check GREEN (599/76/76) | SFT/GRPO data; leakage-proof splits |
| Faithfulness panel | 14 seats, live; swapped-pair canary rejected at 0.99; blind back-translation caught a shared forward-seat confirmation bias (`trade_deadlock`: fwd 0.88/0.82 vs. back 0.25) | faithfulness metric (gated, §8.4) |
| Real-skills mining | 923 artifacts, 110 candidate teams, 29 coordination-requiring (task-level filter, evidence-quoted); deterministic path 0/29; careful reading recovers 7/16 assessed — stated coordination lives in orchestration configs, not agent files | `test-real` source; in-the-wild evidence |

unlicensed repository quarantined; 110 candidate teams), the filter identifies **29 teams whose tasks genuinely require multi-party coordination**, every verdict carrying verbatim evidence quotes from the source text. On those 29: the deterministic path still recovers zero; careful evidence-only reading, applied to 16 of the 29, recovers a checker-valid protocol for 7 and surfaces genuine role incompatibilities in 3. The sharpest regularity is *where* recovered structure lives: per-agent skill files across three independent catalogs describe coordination-requiring tasks in only 2–7% of groupings, while orchestration configurations state the flow essentially whenever they are present at all—coordination, when written down, lives in the orchestrator layer, not in the per-agent artifact, which is precisely the layer this compiler formalizes. Two scope notes keep this honest: the measured population is what authors *write down*, not what deployed systems *need* (a low rate in artifacts implies nothing about demand for coordination); and mined data still yields too few `test-real` items to stand alone: of the recovered protocols, four trace to permissively licensed upstream repositories and are releasable, while a fifth traces to a repository that carries no license and is therefore withheld [44]. The human-read baseline is packaged and pending.

## 8.4 Preregistered gates, and the results template

The training outcomes are governed by six go/no-go predictions written before any run, labeled H1–H6 ("H" for hypothesis), in the same discipline as the $n$=100 suite (§6.7). Each hypothesis pins down, in advance, one design decision the training program depends on: what would have to be true for that decision to survive, and what happens if it does not. None has been graded yet—the GPU runs are the declared next step—so what follows is the contract the runs will be held to, not a result.

- **H1 — does forcing grammatical output help or hurt?** Grammar-constrained decoding guarantees every draft parses, but recent evidence shows such constraints can degrade the *content* of what is generated, not just its format [42]. H1 therefore runs constrained decoding on vs. off as a primary two-sided test at both training phases; if the constraint costs more than two points of semantic validity or graded equivalence, it is demoted from generation-time constraint to data-generation filtering.
- **H2 — how far does sampling alone go?** (A measurement, not a gate.) Before training any weights, draw $n$ samples from an off-the-shelf model and keep the first that passes the validator. H2 predicts this lifts validity@$k$ well above validity@1; it establishes the prompt-only baseline any trained model must beat, and fills the pending E5 cells.
- **H3 / H4 — did the two training phases actually improve the translator?** H3 gates the supervised fine-tuning (SFT) checkpoint and H4 the reinforcement-learning (GRPO) checkpoint, on validity, graded equivalence to the reference protocol, and repair rounds needed. H4 is written headroom-aware: if H3 already drives validity

**Table 11:** T0—the untrained, prompt-only baseline (no weights trained): validated best-of-$n$ over three API tiers. Template; all cells are macros.

| Metric | Haiku | Sonnet | Opus |
|---|---|---|---|
| validity@1 | PENDING | PENDING | PENDING |
| validity@10 | PENDING | PENDING | PENDING |
| repair-rounds | PENDING | PENDING | PENDING |
| $-to-accepted | PENDING | PENDING | PENDING |

**Table 12:** Trained systems and panel calibration. Template; all cells are macros, PENDING until the GPU runs land. AUC = area under the receiver-operating-characteristic curve.

| Measure | Result |
|---|---|
| SFT-7B validity@1 (test-syn) | PENDING |
| SFT-7B bisim@1 (equiv. to gold) | PENDING |
| SFT-7B $-to-accepted | PENDING |
| GRPO $\Delta$ validity@1 vs. SFT | PENDING |
| GRPO $\Delta$ graded-equiv.@1 | PENDING |
| GRPO $\Delta$ repair-rounds | PENDING |
| Panel AUC (gold vs. mutant) | PENDING |
| Panel human-agree (Wilson LB) | PENDING |
| Panel swapped-pair rejection | PENDING |
| Panel effective votes | PENDING |
| Transfer gap ($\Delta$ of gaps) | PENDING |

near its ceiling, H4's primary metric switches to graded equivalence rather than demanding an arithmetically impossible further validity gain.

- **H5 — can the faithfulness judge panel be trusted?** The panel may not reward or gate anything until it demonstrably agrees with human judgment. The statistics are honest about power: certifying an 85% agreement threshold would need on the order of 370 human-labeled items, so the gate instead binds on a Wilson 95% lower bound $\geq 0.80$ over $n \geq 200$ items, evaluated on non-circular strata (mined and human-audited items, not the back-translation winners the judging family itself selected).
- **H6 — does any of it transfer to intents real humans write?** The corpus is partly synthetic, so H6 measures the synthetic-to-human generalization gap on human-written intents, reported as a bootstrap point estimate with a confidence interval—at $n$=150–300 the gap-of-gaps is underpowered as a binary test, so the interval width is reported, not hidden.

If a gate misses, the program degrades gracefully (SFT ships without GRPO; the panel stays advisory forever) because the verifier rewards never depended on the judge.

Tables 11 and 12 are the template. Every cell is a macro from `seam_results.tex` that currently renders PENDING and is replaced, post-GPU, by editing that one file (`TEMPLATE_HOWTO.md` maps each macro to its harness output field); the table structure never changes, so a compiled cell reads either a measured number or an honest PENDING—exactly as the E5 and E3-vendor cells elsewhere in this paper do.

**Positioning and verified novelty.** The recipe's lineage is explicit—RL from verifiable rewards [36, 37, 38, 39], expert iteration [35], back-translation and verifier-checked autoformalization [40, 41], and independent judge panels over a single judge [45] with the 2026 evidence that panels carry far fewer effective votes than seats [46] and need a robust (geometric-median) aggregator [47], both of which this panel adopts. The task itself, however, appears to be new: nine query variants across the MPST/Scribble/choreography vocabulary crossed with LLM/NL-generation phrasing returned no prior system translating natural-language intent into multiparty session types. Two near-misses are named to pre-empt the comparison: ZipperGen [12] shares the projection-to-deadlock-free-endpoints philosophy but has no natural-language front end (it is a programmer-authored MSC DSL), and Liu et al. [48] shares the two-stage NL→formal-spec shape but targets network I/O grammars for test generation, with no global type, projection, or well-formedness dimension. Neither anticipates a global-type-with-projection target drafted from intent.

# 9 Three Typed Extensions, Measured

The validation suite certifies what STJP ships; this section extends what it can *catch*, *build-check*, and *survive*. The three extension experiments continue the suite's numbering as E8–E10, with one prototype per session-type result:

**Table 13:** Scorecard for the three extensions (E8–E10). "Shipped" is the full STJP of §7.

| Extension (theory) | Shipped STJP | With extension | Verdict |
|---|---|---|---|
| E8 session ledger [29] | blind to cumulative overruns: 0/50 detected; live, 16 over-budget debits paid | 50/50 caught at the exact crossing message, 0/50 false positives; live, 0 paid, goal still reached | new guarantee |
| E9a subtype build check [30] | "lean $\leq$ projection" asserted informally | decidable build step; catches a lean contract that drops a required send | hardening |
| E9b tolerant gate [30, 31] | strict gate | 0/50 illegal sends admitted; 0 anticipations needed live | safe; boundary result |
| E10 crash handling [49] | 100% limbo on crash (14/14 grid; 19/19 replays; 15/15 live) | 100% typed terminal, 0 disasters; 5/5 unsafe handlers rejected statically | new checked property |

guards [11] $\rightarrow$ stateful properties [29] $\rightarrow$ lawful slack [30, 31] $\rightarrow$ typed recovery [49]. Each prototype passed a hand-derived verdict corpus (12/14/12 traces) before its benchmark ran; all predictions were pre-registered and graded, including the one we grade against ourselves. These are failure-mode coverage additions, not speedups: none moves the core cost or safety numbers of §6–§7, and each write-up names the shipped system's blindness as the motivation, not a planted defect.

**E8 — Stateful session invariants (the ledger).** Per-message guards see one message; a budget of \$10k with a per-debit limit of \$5k is overrun by three \$4k debits that every shipped check individually approves—a violation class the gate is blind to *by construction*. Following Chen & Honda [29], the `.refn` sidecar gains `state`/`on`/`invariant` clauses; the generated monitor carries a session store, applies updates on accept, and checks invariants before delivery (centrally at the gate in v1, deferring the per-endpoint distribution—where the asynchronous machinery of Chen & Honda [29] becomes essential—to future work). Measured: on 100 seeded traces, the shipped configuration detects 0/50 overruns, the ledger flags 50/50 *at the exact crossing message* with 0/50 false positives; in live trials (a \$12k procurement against the \$10k budget, $n$=15/configuration) the shipped gate paid 16 over-budget debits undetected, the ledger-observe configuration flagged 15/15, and the ledger *gate* paid exactly \$10,000 in every trial while still reaching the goal—the Treasurer, re-prompted with the invariant text, downsized or refused. Cost: a few sidecar clauses. Bounds: invariants are checked dynamically (not SMT-proved) and centrally.

**E9 — Lawful slack (precise subtyping), a win and a boundary.** Two deliverables from one theorem. *(2a)* A coinductive subtype checker (output covariance, input contravariance, send-polarity) makes lean $\leq$ projection a decidable *build step*: contract compression is now verified rather than asserted, and the checker catches a seeded lean contract that drops a required send. *(2b)* A tolerant gate implementing the one decidable, provably safe relaxation—independent-receive output anticipation—behind a default-off flag. The mandatory control holds: 0/50 illegal sends admitted, so tolerance never widens the gate beyond the precise relation. The deadlock replay, pre-registered as uncertain ("$\geq$5/19 rescued"), returned the more interesting answer: 16/19 deadlocks do contain a rejected send that subtyping proves type-safe (the strict gate *is* stricter than safety requires), yet all 16 share one root cause—the Buyer never sent `Deposit`; the rejected sends were other roles correctly waiting on an absent participant, so admitting them rescues nothing—and live reruns needed 0 anticipations ($n$=15/configuration, both gates 15/15). The boundary result is worth stating plainly: on authorization-laden protocols, message orderings *encode obligations* (pay-before-ship), so the slack that subtyping licenses at the type level is precisely what one does not want to spend; lawful slack is real, and this domain is where you leave it unspent. The type theory and the deployment answer diverge, and reporting both is the point.

**E10 — Typed crash handling (no session left in limbo).** The audit's 22 non-terminal trials were untyped crash failures; following Viering et al. [49], a `.fail` sidecar declares per-region handlers, the validator statically checks *coverage* (every state $\times$ crashable role handled or explicitly escalated), *handler safety* (handler bodies are re-validated as mini global types through the same Scribble pipeline), and *recoverability* (every crash reaches a typed terminal); the scheduler—already the coordinator—gains a poll-timeout crash detector, and monitors swap to pre-generated handler EFSMs. Measured: the full crash-point grid (14 cells) leaves the shipped system in limbo 14/14, while STJP+CF (STJP extended with the crash-failure handling above) reaches 14/14 typed terminals (9 degraded, e.g. `Refunded`, 5 clean aborts) with 0 disasters; all 19 real recorded deadlocks replay to typed terminals; live flaky-role trials ($n$=15, one role goes silent per trial) resolve 15/15. Critically, the new checker is held to the E1 preciseness

standard: 5/5 seeded unsafe handlers are rejected statically—including a settlement-shortcut handler whose "recovery" would bypass authorization, exactly the class a naive recovery mechanism would smuggle in. "No session left in limbo" is now a compile-time property, and the audit finding that embarrassed us most is the one this section closes.

# 10 Limitations

**The intent-to-protocol translation step.** Intent→Scribble and contract→skill rendering pass through an LLM; the validator rejects structurally invalid drafts, but a valid-yet-unintended protocol survives (validated $\neq$ intended). Shipped mitigations: human endorsement of G; the outcome anchor. Underway: §8 realizes the seam as a verifier-in-the-loop learning program—validated grammar, EFSM-equivalence scorer, EFSM-deduplicated corpus, and a calibration-gated faithfulness panel, all measured before training—with best-of-$n$ validated sampling as the training-free baseline and back-translated corpus pairs as the fine-tuning path; the GPU training outcomes are preregistered (H1–H6) and pending, entering the paper through the §8 results template. **Scope of the theorems.** T3 is per-session; interleaved sessions need compositional/hybrid extensions; dynamic role join/leave needs [33]; our experiments fix the role set. **Expressiveness.** Open-ended brainstorming resists protocolization, and forcing it into a type would be dishonest; STJP targets the coordination-critical, authorization-laden regime. Probabilistic session types are a candidate middle path. **Statistics and scope of the $n$=100 suite.** The live gpt-4o/gpt-5.4 finance runs remain $n \leq 10$ per cell; the $n$=100 suite fixes the statistical power but substitutes cheap subagent role-players (ladders) or contract-following strategies (E4) for frontier models, and plays each trial's roles from one subagent's per-role views rather than truly independent agents; the nine real-skills live runs (§7) are $n$=10 per configuration with round-batched role-play, so same-role trials within a round share one model context and are not fully independent. The E3 sweep now spans three Claude tiers on both tasks but lacks a non-Claude frontier point (an access limitation), so vendor generality of the capability curve remains open; the tone finding (§6.8) especially needs cross-model replication. At $n \leq 10$ the Wilson 95% intervals are wide enough that adjacent configurations in Tables 3–5 overlap; only the largest gaps (bare vs. enforced) clear them, and headline claims should be re-run at $n \geq 30$. **Scoring-rule history.** The finance tables (§6) were scored under the pre-audit strict label rule, which the fairness audit found unfair to configurations never shown the vocabulary (§6.5); bare-configuration completion there is a lower bound, and per-configuration-rule re-runs are pending. Wall-clock figures from the pre-audit contended runs are labeled indicative and carry no comparative claim. **No tools; invented data.** Roles carry no tools or files: every payload is invented by the model on the spot, so the C1 provenance check currently certifies message discipline, not real data lineage; a variant in which a tool actually serves the revenue figure is the experiment that would make the provenance claim solid. **Headline runs ride one deployment.** The finance headline runs used a single Azure deployment named `gpt-5.4`, a user-set deployment name whose underlying model we cannot independently verify; the Claude-tier ladders diversify the model axis, but the headline finance numbers are one-deployment results. **Grader and guards.** The post-hoc severity grader is payload-blind at milestones (`Approval(False)` currently counts as the authorization milestone; the monitor-level guard closes this, the grader upgrade is pending); choice-guard authoring is manual (the drafter should co-emit the sidecar). **Extension bounds.** The crash-handling roadmap item is now implemented and measured (§9), with its own bounds: a `.fail` sidecar must be authored per protocol, and the live crash model is a role going silent. The session ledger checks invariants dynamically and centrally, not by SMT proof or per-endpoint distribution [29]; the tolerant gate is provably safe but, on obligation-encoding protocols, deliberately left off by default (§9). **Monitor correctness** is validated (theory-matching, regression tests, manual audits), not mechanized.

# 11 Conclusion

When natural-language coordination artifacts—skill files, agent definitions, specification documents—became the programming language of multi-agent systems, the field skipped the compiler and went straight to observability. STJP retrofits the missing stage: intents compile into a protocol algebra with a forty-year pedigree; faulty choreographies are rejected before a single token is spent; and the artifacts agents actually receive—lean local skills, generated monitors, guards, a protocol-driven scheduler—are projections of a proven object rather than informal approximations of a hopeful one. A 1,200-trial validation suite replicates the shape at $n$=100: observing configurations accumulate real violations that goal-rate metrics hide, enforcing configurations show zero by construction, and the checker itself survives mutation testing with zero false positives. Nine cases built from unmodified public skill files replicate the shape

in the wild and add its sharpest form: on a looping review protocol the plan-as-text configuration livelocks through its entire budget at $3.6\times$ the cost of the compiled configuration that ships—the failure costs more than the guarantee. The empirical shape is what compilation always buys: the same guarantees, then made cheap ($9\times$ on cost-to-goal once the projection is allowed to *drive* the runtime), plus a class of properties—deadlock-freedom, goal reachability, authorization ordering, guarded branching, recipient confidentiality—that no amount of runtime watching can provide, because they quantify over runs that never happen. Prompts move probabilities; only checks move guarantees. The open seam—natural language in, formal protocol out—is real, measurable, and, we argue, the right next benchmark for the field.

# References


[1] Anthropic. Agent Skills open standard (Dec 2025), https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills (reference implementation: https://github.com/anthropics/skills); AGENTS.md convention, https://agents.md/ (https://github.com/agentsmd/agents.md); OpenAI Model Spec, https://model-spec.openai.com/ (https://github.com/openai/model_spec); Model Context Protocol specification (2025-11-25), https://modelcontextprotocol.io/specification/2025-11-25 (https://github.com/modelcontextprotocol/modelcontextprotocol).

[2] S. Saha, P. Hemanth. Skilldex: a package manager and registry for agent skill packages with hierarchical scope-based distribution. arXiv:2604.16911, 2026.

[3] H. Wang, C. Poskitt, J. Sun. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE* 2026; arXiv:2503.18666.

[4] Agent-C: enforcing temporal constraints for LLM agents. arXiv:2512.23738, 2025.

[5] I. Levy, B. Wiesel, S. Marreed, A. Oved, A. Yaeli, S. Shlomov. ST-WebAgentBench: evaluating safety and trustworthiness in web agents. *ICLR* 2026; arXiv:2410.06703.

[6] X. Wei et al. BeSafe-Bench: unveiling behavioral safety risks of situated agents in functional environments. arXiv:2603.25747, 2026.

[7] Surveys of LLM-as-judge reliability: positional/verbosity/self-preference biases; TNR $<25\%$; expert agreement 64–68%. Masood 2026; Deepchecks 2025; Autorubric, arXiv:2603.00077.

[8] K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *J. ACM* 63(1), 2016.

[9] A. Scalas, N. Yoshida. Less is more: multiparty session types revisited. *POPL* 2019.

[10] N. Yoshida, R. Hu, R. Neykova, N. Ng. The Scribble protocol language. *TGC* 2013, LNCS 8358, Springer, 2014. Reference toolchains: scribble-java, nuScr, mpstk.

[11] L. Bocchi, K. Honda, E. Tuosto, N. Yoshida. A theory of design-by-contract for distributed multiparty interactions. *CONCUR* 2010.

[12] B. Bollig, M. Függer, T. Nowak. Provable coordination for LLM agents via message sequence charts. arXiv:2604.17612, 2026. ZipperGen; results machine-checked in Lean 4.

[13] T. Liu. Contractual skills: a GovernSpec design framework for enterprise AI agents. arXiv:2605.22634, 2026.

[14] Y. Ge. Governance architecture for autonomous agent systems: threats, framework, and engineering practice (LGA). arXiv:2603.07191, 2026.

[15] M. Kaptein et al. Runtime governance for AI agents: policies on paths. arXiv:2603.16586, 2026.

[16] L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, N. Yoshida. Monitoring networks through multiparty session types. *FORTE* 2013; *TCS* 2017.

[17] F. Montesi. *Introduction to Choreographies*. Cambridge University Press, 2023.

[18] M. Carbone, F. Montesi. Deadlock-freedom-by-design: multiparty asynchronous global programming. *POPL* 2013.

[19] M. Cosler et al. NL2Spec: interactively translating unstructured natural language to temporal logics with LLMs. *CAV* 2023.

[20] Sharma et al. PROSPER: extracting protocol specifications using LLMs. *HotNets* 2023.

[21] A. Igarashi, P. Thiemann, Y. Tsuda, V. Vasconcelos, P. Wadler. Gradual session types. *ICFP* 2017; *JFP* 2019.

[22] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete multiparty session type projection with automata. *CAV* 2023.

[23] E. Li, F. Stutz, T. Wies, D. Zufferey. Characterizing implementability of global protocols with infinite states and data. *Proc. ACM Program. Lang.* 9(OOPSLA1), 2025.

[24] E. Li, T. Wies. Certified implementability of global multiparty protocols. *ITP* 2025.

[25] C. Paduraru, P.-L. Bouruc, A. Stefanescu. A trace-based assurance framework for agentic AI orchestration: contracts, testing, and governance. arXiv:2603.18096, 2026.

[26] Chen et al. TRACE: trajectory-aware comprehensive evaluation. *WWW* 2026.

[27] P.-M. Deniélou, N. Yoshida. Multiparty session types meet communicating automata. *ESOP* 2012.

[28] Refinements for multiparty message-passing protocols. *ECOOP* 2024.

[29] T.-C. Chen, K. Honda. Specifying stateful asynchronous properties for distributed programs. *CONCUR* 2012, pp. 209–224.

[30] T.-C. Chen, M. Dezani-Ciancaglini, A. Scalas, N. Yoshida. On the preciseness of subtyping in session types. *Logical Methods in Computer Science* 13(2), 2017.

[31] T.-C. Chen, M. Dezani-Ciancaglini, N. Yoshida. On the preciseness of subtyping in session types: 10 years later. *PPDP* 2024, 2:1–2:3.

[32] N. Yoshida, P.-M. Deniélou, A. Bejleri, R. Hu. Parameterised multiparty session types. *FoSSaCS* 2010.

[33] P.-M. Deniélou, N. Yoshida. Dynamic multirole session types. *POPL* 2011.

[34] SafeRL-Lab (SAIL-Lab). AgenticPay: a buyer–seller negotiation benchmark for LLM agents. GitHub, 2025. MIT license. https://github.com/SafeRL-Lab/AgenticPay (commit 9740c5e).

[35] E. Zelikman, Y. Wu, J. Mu, N. D. Goodman. STaR: bootstrapping reasoning with reasoning. *NeurIPS* 2022.

[36] Z. Shao et al. DeepSeekMath: pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024. Group Relative Policy Optimization (GRPO); RL from verifiable rewards.

[37] Q. Yu et al. DAPO: an open-source LLM reinforcement learning system at scale. arXiv:2503.14476, 2025.

[38] C. Zheng et al. Group sequence policy optimization. arXiv:2507.18071, 2025.

[39] Z. Liu et al. Understanding R1-Zero-like training: a critical perspective (Dr. GRPO). arXiv:2503.20783, 2025.

[40] R. Sennrich, B. Haddow, A. Birch. Improving neural machine translation models with monolingual data (back-translation). *ACL* 2016.

[41] Verifier-checked autoformalization: Lean-ing on Quality (arXiv:2502.15795, 2025); Cycle-Consistency Fine-tuning for Lean4 (arXiv:2603.24372, 2026); miniF2F pass@k methodology.

[42] I. Y. Lee, L. D'Antoni, T. Berg-Kirkpatrick. The format tax: constrained/structured decoding can degrade reasoning and writing quality. arXiv:2604.03616, 2026.

[43] D. Banerjee, T. Suresh, S. Ugare, S. Misailovic, G. Singh. CRANE: reasoning with constrained LLM generation. *ICML* 2025; arXiv:2502.09061.

[44] Verified canonical URLs, license files, and exact-file permalinks for every mined real-world skill/example and cited standard referenced above, checked against the live repositories on 2026-07-12: `docs/reference/MINED_SKILLS_SOURCES.md` (this repository); machine-readable companion `experiments/seam_bench/mining/sources.json`.

[45] P. Verga et al. Replacing judges with juries: evaluating LLM generations with a panel of diverse models (PoLL). arXiv:2404.18796, 2024.

[46] G. Kohli. Nine judges, two effective votes: correlated errors undermine LLM evaluation panels. arXiv:2605.29800, 2026.

[47] RoPoLL: robust panel of LLM judges; geometric-median aggregation with a $1/2$ breakdown point. arXiv:2606.30931, 2026.

[48] K. Liu, D. Chakraborty, A. Liggesmeyer, A. Zeller. Synthesizing precise protocol specs from natural language for effective test generation. arXiv:2511.17977, 2025.

[49] M. Viering, T.-C. Chen, P. Eugster, R. Hu, L. Ziarek. A typing discipline for statically verified crash failure handling in distributed systems. *ESOP* 2018, pp. 799–826.

[50] OWASP Foundation. Top 10 for agentic applications, 2026 edition.

# A A validated protocol and its guard sidecar

Global type (Scribble) for the finance case, abbreviated:

```
global protocol Finance(role Ingestor, role RevenueAnalyst, role ExpenseAnalyst,
                        role TaxVerifier, role Auditor, role Writer) {
  RawRevenueData(Payload)  from Ingestor to RevenueAnalyst;
  ExpenseData(Payload)     from ExpenseAnalyst to RevenueAnalyst;
  choice at RevenueAnalyst {
    HighRevenueNotification(Analysis) from RevenueAnalyst to Auditor;
    AuditReport(Findings)             from Auditor        to TaxVerifier;
  } or {
    StandardRevenueNotification(Analysis) from RevenueAnalyst to TaxVerifier;
  }
  Approval(Decision)       from TaxVerifier to Writer;   // goal marker
  FinalReport(Document)    from Writer to Ingestor;      // goal marker, FINAL
}
```

Choice-guard sidecar ( `.refn`; asserted-MPST style; stock Scribble untouched):

```
[choice at RevenueAnalyst]
when:    float(RawRevenueData) > 50000
require: HighRevenueNotification
over:    StandardRevenueNotification

[payload Approval]
assert:  Decision == 'true'  =>  next in {FinalReport}   # denied-but-debited hole
```

The guard travels through four stages: *define* (sidecar) → *state* (compiled into the contract at the decision state) → *check* (value-tracking monitor; verdict `choice_guard_violation`) → *enforce* (gate). Offline replay verification fires on high-revenue+standard-branch and on the symmetric case, stays silent on correct branches and on unevaluable guards.

# B Reproducibility

All configurations, cases, protocols, guards, run directories (events, prompts, per-message verdicts), the severity grader, and the metric engine are in the accompanying repository; every number in §6 names its run directory.

**Layout.** Each case lives in `experiments/cases/<case>/` (Scribble protocol, guard sidecar, goals file, per-configuration prompts, and a `runs/` directory of raw traces). The harness is `experiments/scripts/`: `case_runner.py` (drive a case), `evaluate_run.py` (three-rung scoring), `re_anchor_goals.py --check` (answer-key invariance audit, zero LLM calls), `stats.py` (Wilson intervals), `scaling_chart.py` and `roles_sweep.py` (scaling; the `plot` and proxy paths need no cloud access). Configuration definitions are in `experiments/baselines/`.

**Running.** `python experiments/scripts/case_runner.py <case> <n> --configurations <keys> [--sequential]`. Every summary records which success rule scored each configuration (`success_rule`),

the execution mode (sequential vs. contended parallel), Wilson 95% intervals, and any prompt truncation at the hosting service's 8,000-character limit; every prompt each agent saw is persisted with a checksum.

**Determinism.** The role-players are hosted stochastic models (no seed control is exposed through the hosting APIs), so trials are not replayable; everything downstream of the trace is deterministic—re-running the evaluators on a run directory reproduces every table bit-for-bit. Traces are the artifact of record.

**Cost.** A full finance run (15 configurations $\times$ 10 trials, retries included) is roughly 5M–10M tokens $\approx$ \$20–45 on gpt-4o-class pricing ($\approx$\$2 on a mini-tier model); the two-case scaling run $\approx$ 5M tokens $\approx$ \$15–20; the entire 1,200-trial subagent suite cost under \$100; one-time protocol drafting and goal re-anchoring cost cents. Per-command cost lines: `docs/reference/COST_ESTIMATES.md` in the repository.